Keycloak

Stefan Reuter

Version 3.0, 20.10.2026



Inhaltsverzeichnis

1. Einführung . . . 1
   - Was ist Keycloak? . . . 1
   - Keycloak ist Open Source . . . 1
   - Features . . . 1
   - Informationsquellen . . . 2
   - Releases und Community . . . 2
   - Releaseplanung . . . 2
   - Durchgängiges Übungsszenario . . . 3
2. OAuth 2.0 und OpenID Connect . . . 4
   - Lernziele . . . 4
   - Das Problem . . . 4
   - OAuth 2.0 . . . 4
   - Rollen in OAuth 2.0 . . . 4
   - Authorization Code Flow . . . 4
   - PKCE . . . 5
   - Client Credentials Flow . . . 5
   - OpenID Connect (OIDC) . . . 5
   - Token-Typen . . . 6
   - JSON Web Token (JWT) . . . 6
   - JWT Payload — Standard Claims . . . 6
   - JWT Signatur . . . 6
   - Token-Validierung . . . 7
   - OIDC Discovery . . . 7
3. Installation . . . 8
   - Lernziele . . . 8
   - Architektur im Überblick . . . 8
   - Development Mode . . . 8
   - Exkurs: Docker Compose . . . 9
   - Übung 01: Keycloak starten . . . 9
   - Admin User & Admin Console . . . 9
   - Konfigurationsoptionen . . . 9
   - Eigenschaften Dev Mode . . . 10
   - Production Mode . . . 10
   - Beispiel Prod Mode . . . 10
   - TLS-Zertifikat über ACME erzeugen . . . 11
   - Reverse Proxy . . . 11
   - Traefik als Reverse Proxy . . . 11
   - Übung 02: Traefik . . . 12

Tipps bei Timeout . . . 12
Datenbank . . . 12
PostgreSQL als Container . . . 12
Datenbank konfigurieren . . . 13
Übung 03: PostgreSQL . . . 13
Optimierter Keycloak Build . . . 13
kc.sh build . . . 14
Optimiertes Container Image bauen . . . 14
Übung 04: Discovery erkunden . . . 14
4. Realms . . . 15
Lernziele . . . 15
Was ist ein Realm? . . . 15
Das Realm-Modell . . . 15
Welche Realms gibt es? . . . 15
Realm anlegen . . . 16
Realm konfigurieren . . . 16
Übung 05: Realm anlegen . . . 17
Authentication-Konfiguration . . . 17
Password Policy . . . 17
Antipattern: Erzwungene Passwortänderung . . . 18
Übung 06: Selbstregistrierung . . . 18
5. Benutzer, Rollen & Gruppen . . . 19
Benutzer anlegen . . . 19
User Profile . . . 19
Rollen . . . 19
Zuweisen von Rollen . . . 19
Übung 07: Benutzer mit Rollen . . . 20
Gruppen . . . 20
Typische Nutzung . . . 20
6. LDAP & Active Directory . . . 21
Anbindung . . . 21
Synchronisation . . . 21
Übung 08: Anbindung AD . . . 21
Mapping von LDAP-Attributen . . . 21
LDAP Filter . . . 22
Gruppen & Rollen . . . 22
SCIM . . . 22
LDAP (Pull) vs. SCIM (Push) . . . 22
7. Externe Identity Provider . . . 23
Überblick . . . 23
Ablauf: Identity Brokering . . . 23

Beispiel: GitHub . . . 23
Register Application . . . 23
Client Id & Secret . . . 24
Provider hinzufügen . . . 24
Sign in with GitHub . . . 25
Verknüpfung der Accounts . . . 25
Übung 09: Social Login . . . 26
8. Organisationen . . . 27
Was sind Organisationen? . . . 27
Kernfunktionen von Keycloak Organizations . . . 27
Organisationen aktivieren und anlegen . . . 27
Mitglieder verwalten . . . 27
Identity Provider zuweisen . . . 28
Weiterentwicklung . . . 28
Übung 10: Organisation anlegen . . . 28
9. Clients . . . 29
Lernziele . . . 29
Client anlegen . . . 29
Service Accounts (Client Credentials) . . . 29
Übung 11: Public Client . . . 29
Konfiguration auf Client-Seite . . . 30
Client Scopes & Mapper . . . 30
Autorisierung in der Anwendung . . . 30
Logout . . . 30
Client Policies & FAPI . . . 31
FAPI-Bausteine: PAR & Holder-of-Key . . . 31
PAR & Holder-of-Key: Trade-offs . . . 31
Grenze: Aufrufketten . . . 32
Client Secret Rotation . . . 32
Client Registration Service . . . 32
10. Mehrfaktorauthentifizierung . . . 33
Lernziele . . . 33
MFA . . . 33
Recovery Codes . . . 33
TOTP/HOTP . . . 33
Übung 12: TOTP . . . 33
WebAuthn . . . 34
Authentication Flows . . . 34
Übung 13: WebAuthn . . . 35
Individuelle Lösungen . . . 35
11. Themes . . . 36

Organisation von Themes . . . 36
Theme Types . . . 36
Woraus besteht ein Theme? . . . 36
Entwicklung von Themes . . . 36
theme.properties . . . 37
Beispiel: Custom CSS für Login . . . 37
Übung 14: Theme . . . 37
Tipps & Tools . . . 38
12. Plugins . . . 39
Erweiterung von Keycloak . . . 39
Tipps zur Entwicklung . . . 39
Service Provider Interfaces . . . 39
Provider Info . . . 39
Authentication SPI . . . 40
Action Token Handler SPI . . . 40
Event Listener SPI . . . 40
Protocol Mapper SPI . . . 41
Vault SPI . . . 41
Erweiterungen des Servers . . . 41
13. REST API für Administration . . . 42
Admin CLI . . . 42
REST API . . . 42
Praxis: Benutzerliste für eine Anwendung . . . 42
Export und Import . . . 43
Übung 15: Admin REST API . . . 43
14. Anpassungen für Produktivbetrieb . . . 44
Lernziele . . . 44
Tipps . . . 44
Backup & Restore . . . 44
Health-Endpunkte . . . 44
Metriken und Monitoring . . . 45
Logging . . . 45
Feature Flags . . . 46
Brute-Force-Schutz . . . 46
Security-Hardening-Checkliste . . . 46
Häufige Fehler & Troubleshooting . . . 46
Kubernetes . . . 47
15. Anhang: Betrieb im Cluster . . . 48
Keycloak Cluster . . . 48
Caches . . . 48
Konfiguration der Caches . . . 48

Kubernetes Transport Stack . . . 48
Installation in Kubernetes . . . 49
Multi-Site Deployments . . . 49
Multi-Cluster v2 (stateless) . . . 50
16. Anhang: Cheat-Sheet . . . 51
Wichtige Endpunkte . . . 51
Wichtige Umgebungsvariablen . . . 51
kcadm.sh — Spickzettel . . . 51
17. Lösungsvorschläge . . . 53
Lösung zu Übung 01 . . . 53
Lösung zu Übung 02 . . . 53
Lösung zu Übung 03 . . . 53
Lösung zu Übung 04 . . . 53
Lösung zu Übung 05 . . . 54
Lösung zu Übung 06 . . . 54
Lösung zu Übung 07 . . . 54
Lösung zu Übung 08 . . . 54
Lösung zu Übung 09 . . . 54
Lösung zu Übung 10 . . . 54
Lösung zu Übung 11 . . . 55
Lösung zu Übung 12 . . . 55
Lösung zu Übung 13 . . . 55
Lösung zu Übung 14 . . . 55
Lösung zu Übung 15 . . . 55

Kapitel 1. Einführung

Was ist Keycloak?

- Single-Sign-On-Lösung für Webanwendungen und RESTful Web-Services
- Zentrale und eigenständige Komponente in der Anwendungslandschaft für Authentifizierung
- Anpassbares UI für Login, Registrierung, Administration und Account-Verwaltung
- Kann an LDAP oder Active Directory angebunden werden
- Unterstützt Delegation der Authentifizierung z.B. an Facebook oder Google ("Login mit Facebook")

Keycloak ist Open Source

- Keycloak ist Open-Source-Software unter der Apache Lizenz
- Incubation Projekt bei der Cloud Native Computing Foundation
- Support vom Open-Source-Projekt gibt es jeweils **nur** für die letzte veröffentlichte Version
- Kommerzieller Support erhältlich z.B. als Red Hat build of Keycloak und von unabhängigen Anbietern



Features

- Single-Sign On und Single-Sign Out für Browser-Anwendungen (Authentifizierung)
- Support für OpenID Connect und OAuth 2.0
- Support für SAML
- Identity Brokering: Unterstützung externer OpenID Connect oder SAML IdPs

- Social Login (Google, GitHub, Facebook, etc.)
- User Federation: Synchronisation der Benutzer mit LDAP oder AD
- Kerberos Bridge
- Admin Console
- Account Management Console für Benutzer
- Anpassbare Themes
- 2FA: TOTP/HOTP über Google Authenticator oder FreeOTP
- Unterschiedliche Login-Flows: Registrierung, Passwort vergessen, E-Mail-Adresse bestätigen, erzwungenes Passwort-Update, …
- Session Management: Admins und Benutzer können aktive Sessions sehen und verwalten
- Token Mapper, um Attribute von Benutzern, Rollen, etc. in Tokens zu mappen

Informationsquellen

- Keycloak Homepage: https://www.keycloak.org/
- Offizielle Dokumentation
  - Server Administration
  - Server Developer für Themes und Plugins
  - Upgrading Guide
  - Admin REST API
- Guides
- Ankündigungen im Blog

Releases und Community

- GitHub Organisation mit Quellcode
- Releases, Nightly Build und geplante Milestones
- Container Images bei Quay.io
  - `latest`, `nightly` und Tags mit spezifischer Versionsnummer
- Release Notes
- Mailing-Listen keycloak-user und keycloak-dev
- GitHub Diskussionsforen

Releaseplanung

- **Vor Version 26**
  - Ca. **4 Major-Releases** pro Jahr
  - Steigende Anzahl von breaking Changes

    - Hoher Aufwand Keycloak-Installationen up-to-date zu halten
- **Seit Version 26**
    - Geplant sind **4 Minor-Releases** pro Jahr
    - Ein Major-Release alle 2 bis 3 Jahre
    - Client-Bibliotheken sollen separat versioniert werden
    - Breaking Changes bei Minor-Releases sollen Opt-In sein

Durchgängiges Übungsszenario

- In den Übungen bauen wir Schritt für Schritt ein vollständiges IAM auf
- Der Realm `training` ist dabei der rote Faden:
    - Realm & Selbstregistrierung einrichten
    - Benutzer, Rollen und Gruppen anlegen
    - Externe Benutzer anbinden (LDAP/AD, Social Login)
    - Anwendungen als Clients absichern
    - Mehrfaktor-Authentifizierung aktivieren
    - Aussehen (Theme) anpassen und per API verwalten

Kapitel 2. OAuth 2.0 und OpenID Connect

Lernziele

- OAuth 2.0 und OpenID Connect unterscheiden können
- Den Authorization Code Flow mit PKCE erklären
- Access-, Refresh- und ID-Token sowie den JWT-Aufbau verstehen

Das Problem

- Anwendungen müssen wissen: **Wer ist der Benutzer?** (Authentifizierung) und **Was darf er?** (Autorisierung)
- Ohne gemeinsamen Standard: jede Anwendung baut eine eigene Login-Lösung
- Passwörter werden direkt an Drittanwendungen weitergegeben — kein gezielter Entzug von Zugriffsrechten möglich

OAuth 2.0

- Offener Standard zur **delegierten Autorisierung** (RFC 6749)
- Ermöglicht es einer Anwendung, im Auftrag eines Benutzers auf Ressourcen zuzugreifen — ohne dessen Passwort zu kennen
- Beispiel: Eine App darf Kalendereinträge lesen, aber keine E-Mails

Rollen in OAuth 2.0

Resource Owner
: Der Benutzer, dem die Ressource gehört

Client
: Die Anwendung, die Zugriff auf die Ressource anfragt

Authorization Server
: Stellt nach erfolgreicher Authentifizierung Tokens aus (z.B. Keycloak)

Resource Server
: Schützt die Ressource und akzeptiert Tokens (z.B. eine REST-API)

Authorization Code Flow

1. Browser ruft geschützte Seite beim Client auf
2. Client leitet den Browser zum Authorization Server um
3. Benutzer meldet sich am Authorization Server an

4. Authorization Server leitet den Browser mit einem **Authorization Code** zurück zum Client
5. Client tauscht den Authorization Code (+ Client Secret) gegen Tokens ein
6. Client nutzt das **Access Token** für Anfragen an den Resource Server

PKCE

- **P**roof **K**ey for **C**ode **E**xchange (RFC 7636)
- Schützt den Authorization Code Flow gegen Abfangen des Authorization Codes
- Vom Standard für öffentliche Clients (Browser-Apps, Mobile Apps) ohne Client Secret gefordert — in Keycloak am Client (Advanced: Code Challenge Method = `S256`) oder per Client Policy zu erzwingen
- Client erzeugt ein zufälliges `code_verifier`-Geheimnis und schickt davon den Hash (`code_challenge`) mit dem Authorization Request mit
- Der Authorization Server merkt sich die `code_challenge` zum ausgestellten Authorization Code
- Beim Einlösen des Codes am Token-Endpunkt schickt der Client den ursprünglichen `code_verifier` mit
- Der Server bildet daraus erneut den Hash und vergleicht ihn mit der gespeicherten `code_challenge` — nur bei Übereinstimmung werden Tokens ausgestellt
- Fängt ein Angreifer den Authorization Code ab, kann er ihn ohne den `code_verifier` nicht einlösen — der Code allein ist wertlos

Client Credentials Flow

- Für **Machine-to-Machine**-Kommunikation ohne menschlichen Benutzer
- Client authentifiziert sich direkt mit Client ID und Secret am Authorization Server
- Erhält ein Access Token für den Zugriff auf eine API
- Typischer Use-Case: Backend-Dienst ruft eine geschützte REST-API auf

OpenID Connect (OIDC)

- Erweiterung von OAuth 2.0 um **Authentifizierung** (OpenID Foundation, 2014)
- OAuth 2.0 sagt: „Dieser Token erlaubt Zugriff auf X" — beantwortet aber nicht „Wer ist angemeldet?"
- OIDC ergänzt:
  - **ID Token**: Informationen über den authentifizierten Benutzer
  - **UserInfo Endpoint**: Abfrage weiterer Benutzerattribute
  - **Discovery**: Standardisierte Beschreibung aller Endpunkte

Token-Typen

Access Token

Kurzzeitiger Token für den Zugriff auf den Resource Server. Wird im `Authorization`-Header mitgeschickt: `Bearer <token>`

Refresh Token

Langlebiger Token zum Erneuern des Access Tokens, ohne den Benutzer erneut anzumelden

ID Token

Enthält Informationen über den authentifizierten Benutzer (nur OIDC). Für die Anwendung, nicht für den Resource Server bestimmt

JSON Web Token (JWT)

- Kompaktes, URL-sicheres Format für Token (RFC 7519)
- Drei Base64url-kodierte Teile, getrennt durch `.`:

```
<Header>.<Payload>.<Signature>
```

Header

Algorithmus (`alg`) und Token-Typ (`typ`)

Payload

Claims — strukturierte Aussagen über den Benutzer und den Token

Signature

Kryptografische Signatur zur Integritätsprüfung

JWT Payload — Standard Claims

Claim	Bedeutung
`sub`	Subject — eindeutige Benutzer-ID
`iss`	Issuer — URL des Authorization Servers
`aud`	Audience — für wen der Token bestimmt ist
`exp`	Expiration — Ablaufzeit (Unix Timestamp)
`iat`	Issued at — Ausstellungszeit
`email`, `name`	Benutzerattribute (OIDC)

JWT Signatur

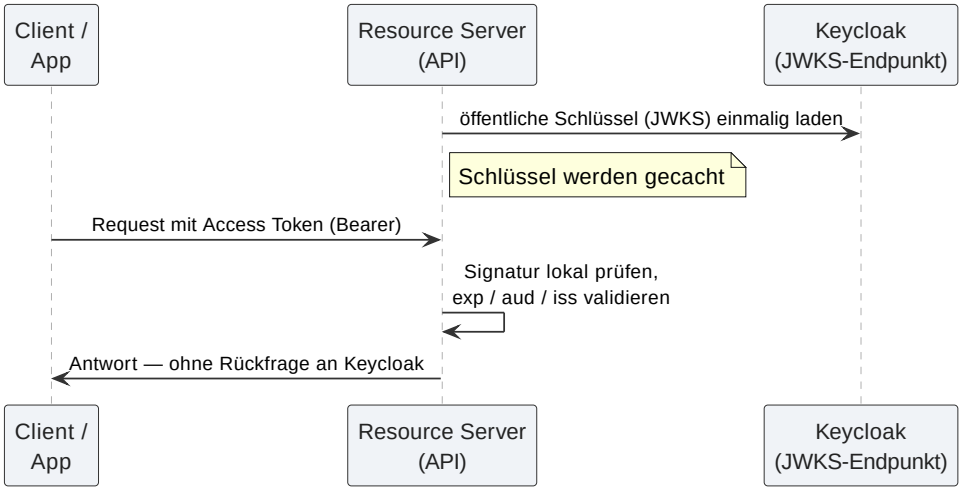
- Der Authorization Server signiert den Token mit seinem privaten Schlüssel

- Resource Server prüfen die Signatur mit dem öffentlichen Schlüssel des Authorization Servers
- Der öffentliche Schlüssel wird über den JWKS-Endpunkt bereitgestellt
- Durch die Signatur kann ein Resource Server einen Token validieren, **ohne den Authorization Server zu befragen**



Tokens lassen sich unter jwt.io oder token.dev dekodieren und inspizieren.

Token-Validierung



OIDC Discovery

- Jeder OIDC-konforme Authorization Server veröffentlicht seine Konfiguration unter einem standardisierten Endpunkt
- Unter Keycloak:

```
/realms/<realm>/.well-known/openid-configuration
```

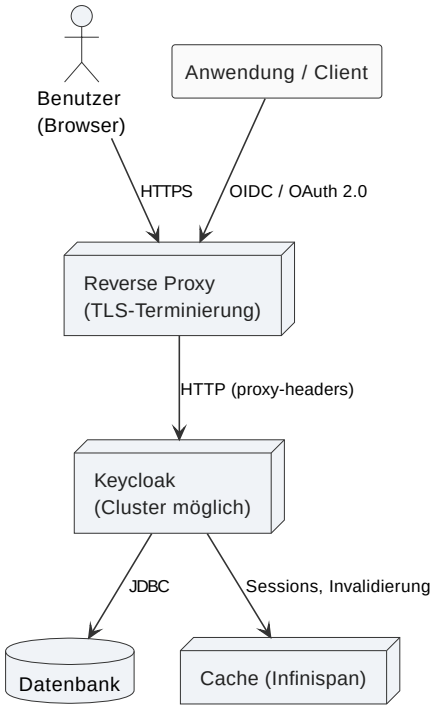
- Enthält u.a. alle Endpunkt-URLs, unterstützte Algorithmen, JWKS-URI
- Die praktische Erkundung des Discovery-Endpunkts folgt als Übung im Kapitel Installation, sobald eine Keycloak-Instanz läuft

Kapitel 3. Installation

Lernziele

- Keycloak im Dev- und Prod-Mode mit Docker starten
- TLS, Reverse Proxy und Datenbank korrekt konfigurieren
- Admin-Bootstrap und wichtige Konfigurationsoptionen kennen

Architektur im Überblick



Development Mode

- Einfachste Möglichkeit Keycloak zu starten, Command: `start-dev`
- Username und Passwort für Administrator über Umgebungsvariablen setzen
- HTTP Port 8080 mappen und von außen verfügbar machen
- `docker-compose.yml`:

```
---
services:
  keycloak:
    image: quay.io/keycloak/keycloak:26.8.1
    # image: registry.redhat.io/rhbk/keycloak-rhel9:26.4
    command: start-dev
    environment:
      KC_BOOTSTRAP_ADMIN_USERNAME: admin
      KC_BOOTSTRAP_ADMIN_PASSWORD: admin
```

```
    ports:
      - "8080:8080"
```

Exkurs: Docker Compose

- Starten über `docker compose up` bzw. `docker compose up -d`
  - Kommandozeilenoption `-d` startet den Container im Hintergrund
- Logs ansehen: `docker compose logs`
- Restart: `docker compose restart`
- Beenden und Löschen: `docker compose down`

Übung 01: Keycloak starten

- Erstelle ein neues Verzeichnis `uebung01` und lege darin eine Datei `docker-compose.yml` an, mit der du Keycloak im Development Mode starten kannst.
- Starte Keycloak und rufe die Oberfläche im Webbrowser auf.
- Melde dich mit dem konfigurierten Admin-Account an der Admin-Console an.

Admin User & Admin Console

- Admin-User und Admin-Passwort können interaktiv über Browser festgelegt werden, wenn Keycloak direkt über "localhost" erreichbar ist (nicht mit Docker).
- Alternativ: über Umgebungsvariablen definieren (`KC_BOOTSTRAP_ADMIN_USERNAME` und `KC_BOOTSTRAP_ADMIN_PASSWORD`).
  - Hinweis: Die früheren Variablen `KEYCLOAK_ADMIN` / `KEYCLOAK_ADMIN_PASSWORD` sind seit Keycloak 26 deprecated.
  - Der so angelegte Account ist ein **temporärer Bootstrap-Admin** (wird in der Console entsprechend markiert) und sollte nach dem initialen Setup durch einen regulären Admin ersetzt werden.
- Aufruf der Admin Console über Browser unter `http://localhost:8080/admin`

Konfigurationsoptionen

- Konfigurationsoptionen können über folgende Mechanismen gesetzt werden:
  - Kommandozeilenparameter
  - Umgebungsvariablen
  - Konfigurationsdateien
- Wird Keycloak als Container betrieben, bietet sich die Nutzung von Umgebungsvariablen an
- Details finden sich im Server Guide Configuring Keycloak
- Eine Liste aller Konfigurationsoptionen stellt das Keycloak-Projekt unter

https://www.keycloak.org/server/all-config bereit

Eigenschaften Dev Mode

- Der Development Mode kommt mit folgenden Standardeinstellungen:
  - HTTP auf Port 8080 ist aktiviert (kein TLS)
  - "Strict hostname resolution" ist deaktiviert
  - Caching ist auf "local" gesetzt, d.h. es wird kein verteilter Cache für Hochverfügbarkeit genutzt
  - Caching von Themes und Templates ist deaktiviert

Production Mode

- Starten mit Command `start` statt `start-dev`
- Benötigt TLS
  - Keycloak benötigt TLS-Zertifikat und privaten Schlüssel, falls es TLS selbst terminieren soll
  - Alternativ: Verwendung eines vorgeschalteten Reverse Proxies zum Terminieren von TLS
- Hostname bzw. URL muss über `KC_HOSTNAME` gesetzt werden

Beispiel Prod Mode

- `KC_HOSTNAME` setzt den URL, unter dem Keycloak angesprochen wird (strict hostname resolution)
- `KC_HTTPS_CERTIFICATE_FILE` und `KC_HTTPS_CERTIFICATE_KEY_FILE` geben an, in welchen Dateien sich TLS-Zertifikat und privater Schlüssel befinden

```
services:
  keycloak:
    image: quay.io/keycloak/keycloak:26.8.1
    command: start
    environment:
      KC_HOSTNAME: https://keycloak.local.2qn.org:8443/
      KC_HTTPS_CERTIFICATE_FILE: /certs/cert.pem
      KC_HTTPS_CERTIFICATE_KEY_FILE: /certs/privkey.pem
      KC_BOOTSTRAP_ADMIN_USERNAME: admin
      KC_BOOTSTRAP_ADMIN_PASSWORD: admin
    ports:
      - "8443:8443"
    volumes:
      - ./certs:/certs
```

TLS-Zertifikat über ACME erzeugen

- TLS-Zertifikate für Keycloak lassen sich von einer öffentlichen CA beziehen, die das ACME-Protokoll unterstützt (z.B. Let's Encrypt oder ZeroSSL)
- Das Kommandozeilentool `acme.sh` kann solche Zertifikate einfach erstellen:

```
docker run --rm -it -v "$(pwd)/acme.sh:/acme.sh" -p 80:80 -p 443:443 neilpang/acme.sh\
  --register-account -m zerossl@example.com

docker run --rm -it -v "$(pwd)/acme.sh:/acme.sh" -p 80:80 -p 443:443 neilpang/acme.sh\
  --issue -d keycloak.example.com --standalone
```

- Ergebnis:

```
achme.sh/keycloak.example.com_ecc/ca.cer
achme.sh/keycloak.example.com_ecc/fullchain.cer         # Certificate Chain (cert.pem)
achme.sh/keycloak.example.com_ecc/keycloak.example.com.cer
achme.sh/keycloak.example.com_ecc/keycloak.example.com.conf
achme.sh/keycloak.example.com_ecc/keycloak.example.com.csr
achme.sh/keycloak.example.com_ecc/keycloak.example.com.csr.conf
achme.sh/keycloak.example.com_ecc/keycloak.example.com.key # Private Key (privkey.pem)
```

Reverse Proxy

- Zwei Varianten bei Nutzung von Reverse Proxy:
  - `forwarded`: Der `Forwarded` Header wird gemäß RFC7239 ausgewertet
  - `xforwarded`: Die nicht standardisierten (aber weit verbreiteten) `X-Forwarded-*` Header werden verwendet (z.B. `X-Forwarded-For`, `X-Forwarded-Proto`, `X-Forwarded-Host` und `X-Forwarded-Port`)
- Welche Header verwendet werden, wird über Konfigurationsoption `proxy-headers` festgelegt
- Soll die Kommunikation zwischen Reverse Proxy und Keycloak unverschlüsselt über HTTP erfolgen, muss zusätzlich die Option `http-enabled` auf `true` gesetzt werden
- Details finden sich im Server Guide Using a reverse proxy

Traefik als Reverse Proxy

- Moderner Reverse Proxy, besonders gut geeignet in Umgebungen mit Docker und Kubernetes
- Konfiguration erfolgt über Kommandozeilenparameter, Konfigurationsdateien und Labels an Docker-Containern
- Unterstützt automatisches Redirect von HTTP auf HTTPS
- Kann TLS Zertifikate über ACME-API (RFC 8555) z.B. über Let's Encrypt erzeugen oder ein extern erzeugtes Zertifikat nutzen.

- Fallstricke gibt es je nach eingesetzten Versionen bei der Nutzung von mTLS (traefik#9669, keycloak#17171, keycloak#46395)

Übung 02: Traefik

- Beende den Docker-Container aus Übung 01.
- Setze Keycloak mit vorgeschaltetem Reverse Proxy im Prod Mode auf. Nutze dafür die zur Verfügung gestellte `docker-compose.yml`-Datei und die Traefik-Konfigurationsdatei `tls.yml` im Verzeichnis `uebung02`.
- Passe in der Datei `.env` den Hostnamen von Keycloak an.
- Erzeuge ein TLS-Zertifikat für den Hostnamen deiner Schulungs-VM.
- Kopiere den Private Key nach `certs/privkey.pem` und die Zertifikatskette nach `certs/cert.pem`.
- Starte die Docker-Container und rufe die Admin-Konsole auf. Prüfe, dass TLS mit dem korrekten Zertifikat verwendet wird.

Tipps bei Timeout

- Tritt beim Laden der Admin-Console ein Timeout auf, ist häufig der Hostname bzw. URL (Umgebungsvariable `KC_HOSTNAME`) nicht richtig gesetzt.
- Prüfe insbesondere das Schema `http` vs. `https` und ggf. den Port.
- Beende Keycloak mit `docker-compose down` und führe eine erneute Initialkonfiguration über `docker compose up` durch.

Datenbank

- In der Standardkonfiguration startet Keycloak mit einer H2-Datenbank (`dev-file`), die Daten werden innerhalb des Containers gehalten und gehen verloren, sobald der Container gelöscht wird
- Für den Produktivbetrieb unterstützt Keycloak folgende Datenbanken:
  - MariaDB Server (`mariadb`) und MySQL (`mysql`)
  - Microsoft SQL Server (`mssql`)
  - Oracle Database (`oracle`)
  - PostgreSQL (`postgres`), auch Amazon Aurora PostgreSQL und EDB
- Der Typ der Datenbank wird über den Parameter `db` bzw. die Umgebungsvariable `KC_DB` gesetzt.
- Details finden sich im Server Guide Configuring the database

PostgreSQL als Container

- `docker-compose.yml`:

```
---
services:
# ...
  db:
    image: postgres:18.2
    restart: unless-stopped
    environment:
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: supersecretpassword
    volumes:
      - ./db:/var/lib/postgresql
```

- Zugriff auf Datenbankkonsole über
  `docker compose exec db psql -U keycloak`

Datenbank konfigurieren

- `KC_DB`: Typ der Datenbank, z.B. `postgres`
- `KC_DB_URL`: JDBC-URL, z.B. `jdbc:postgresql://db/keycloak`
- `KC_DB_USERNAME`: Benutzername für den Datenbankzugang
- `KC_DB_PASSWORD`: Passwort für den Datenbankzugang

Übung 03: PostgreSQL

- Beende den Docker-Container aus Übung 02.
- Setze Keycloak mit PostgreSQL als Datenbank auf. Nutze dafür die zur Verfügung gestellte `docker-compose.yml`-Datei im Verzeichnis `uebung03`.
- In der Datei `.env` die Hostnamen von Keycloak anpassen.
- Starte die Docker-Container und prüfe die Admin-Konsole.
- Verbinde dich über die Datenbankkonsole mit der Datenbank und prüfe, dass die Tabellen von Keycloak (z.B. `user_entity`) dort angelegt wurden

Optimierter Keycloak Build

- Beim Starten von Keycloak über `start-dev` oder `start` wird jeweils zusätzlich ein `build` ausgeführt.
- Beim Build werden einige Optimierungen für den Start und die Laufzeit vorgenommen (Quarkus re-augmentation).
- Der Build kann einige Sekunden dauern und verzögert damit den Start, was insbesondere in dynamischen Umgebungen wie Kubernetes unerwünscht ist.
- Lösung: Build explizit vorab durchführen (z.B. in CI-Pipeline) und optimiertes Container-Image publizieren.

kc.sh build

- Bauen mit `kc.sh build <build optionen>`
  - https://www.keycloak.org/server/all-config?f=build
- Starten mit `kc.sh start --optimized <config optionen>`
  - https://www.keycloak.org/server/all-config?f=config

Optimiertes Container Image bauen

- Dockerfile:

```
FROM quay.io/keycloak/keycloak:26.8.1

RUN /opt/keycloak/bin/kc.sh build \
  --db=postgres \
  --health-enabled=true \
  --metrics-enabled=true
```

- Siehe auch Running Keycloak in a container

Übung 04: Discovery erkunden

- Rufe den Discovery-Endpunkt des `master`-Realms deiner laufenden Instanz auf:

```
https://<HOSTNAME>/realms/master/.well-known/openid-configuration
```

- Welche Endpunkte für `authorization_endpoint`, `token_endpoint` und `jwks_uri` sind hinterlegt?
- Rufe die `jwks_uri` auf — welche Schlüssel sind dort hinterlegt?

Kapitel 4. Realms

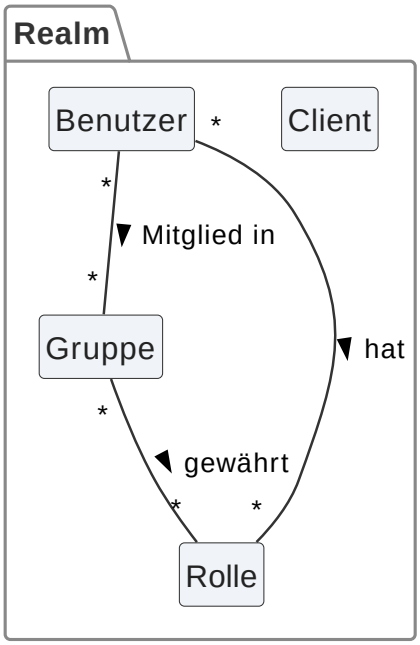
Lernziele

- Zweck und Isolation von Realms verstehen
- Einen Realm anlegen und sinnvoll konfigurieren
- Password Policies und Selbstregistrierung einrichten

Was ist ein Realm?

- Ein Realm verwaltet eine Menge von Benutzern, Credentials, Rollen, Gruppen und Clients.
- Ein Benutzer gehört zu einem Realm und meldet sich an einem Realm an.
- Realms sind **voneinander isoliert** und können nur auf die Benutzer zugreifen, die sie verwalten.

Das Realm-Modell



Welche Realms gibt es?

Master-Realm

Wurde bei der Installation angelegt und beinhaltet den Admin-Benutzer, sollte nur verwendet werden, um andere Realms zu verwalten.

Andere Realms

Werden vom Admin im Master-Realm angelegt und beinhalten die Benutzer, Clients, etc.

Realm anlegen

- Als Admin über die Admin Console im Master-Realm oben links im Drop Down "Create Realm" auswählen und Namen festlegen
- Bei der Planung überlegen, welche Aufteilung sinnvoll ist, da Benutzer und Anwendungen (Clients) jeweils voneinander isoliert sind
- Beispiele:
  - Realm für interne Benutzer (Mitarbeiter), die auf interne Anwendungen zugreifen und Realm für Kunden, die auf Anwendungen für Kunden zugreifen
  - Unterschiedliche Realms für B2B- und B2C-Kunden, wenn diese jeweils unterschiedliche Anwendungen verwenden
  - Realms für unterschiedliche Umgebungen (Dev, Test, Prod)

Realm konfigurieren

- Nach Anlegen des Realms unter "Configure/Realm settings"
- Require SSL: Erzwingt SSL für gar keine, alle oder externe (nicht localhost oder RFC-1918-Adressbereiche) Requests
- Links zu
  - OpenID Endpoint Configuration
  - SAML 2.0 Identity Provider Metadata
- Login-Einstellungen (z.B. Self registration, Passwort vergessen, Remember me)
- Anbindung SMTP-Server für E-Mail-Versand
- Auswahl Themes und Lokalisierung (Festlegen verfügbarer Sprachen, Standardsprache, Texte anpassen)
- Keys
- Event Listener
  - E-Mail: informiert Nutzer über Ereignisse wie Lock-Out oder Passwortänderung
- Events können für Auditzwecke gespeichert werden, optionale Expiration für automatisches Löschen
  - User event settings
  - Admin event settings
- Gültigkeitszeiträume und Timeouts für Sessions und Tokens
- Client Policies
  - Profiles (insbesondere Financial-grade API Security Profile)
  - Policies mit konfigurierten Conditions
- Default Rollen und Gruppen für Benutzer, die über die Registrierung neu angelegt werden

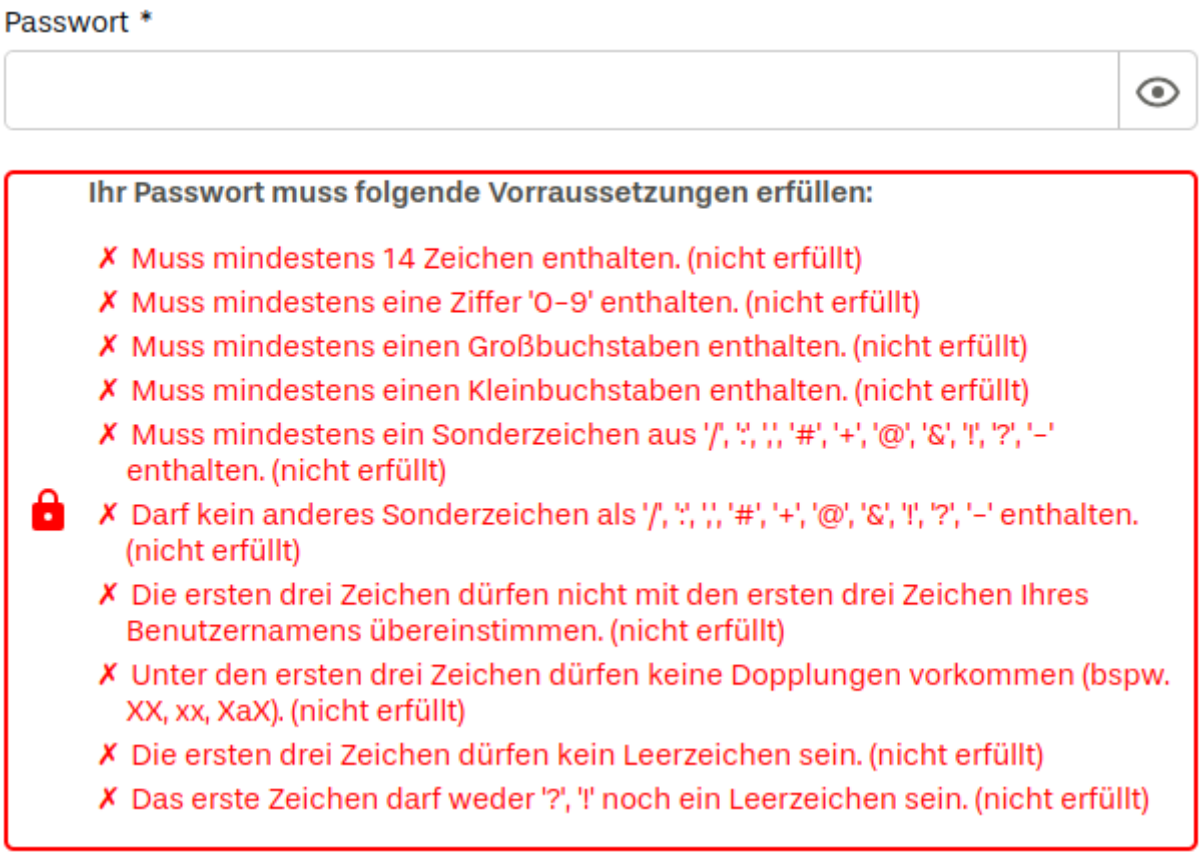
Übung 05: Realm anlegen

- Lege einen neuen Realm `training` an
- Aktiviere die Funktionen "Passwort vergessen" und "Remember me"
- Konfiguriere die E-Mail-Einstellungen für den neuen Realm (Host: `mailcatcher`, Port: `1025`)
- Aktiviere das Speichern von User- und Admin-Events zu Auditzwecken mit einer Haltedauer von jeweils 30 Tagen

Authentication-Konfiguration

- Authentication
  - Flows (Built-in und custom)
  - Required Actions
  - Policies (Password, WebAuthn, OTP)

Password Policy



- Quelle: Zentrale Rechnungseingangsplattform des Bundes (ZRE)
- Humorvolle API: https://passwordpurgatory.com/
- Mindestens aktivieren:
  - Not Username
  - Not Email

- Minimum Length

Antipattern: Erzwungene Passwortänderung

- BSI ORP.4 Identitäts- und Berechtigungsmanagement A23
  - "IT-Systeme oder Anwendungen SOLLTEN NUR mit einem validen Grund zum Wechsel des Passworts auffordern. Reine zeitgesteuerte Wechsel SOLLTEN vermieden werden. Es MÜSSEN Maßnahmen ergriffen werden, um die Kompromittierung von Passwörtern zu erkennen. Ist dies nicht möglich, so SOLLTE geprüft werden, ob die Nachteile eines zeitgesteuerten Passwortwechsels in Kauf genommen werden können und Passwörter in gewissen Abständen gewechselt werden."
- NIST SP800-63B
  - "Verifiers and CSPs SHALL NOT require users to change passwords periodically. However, verifiers SHALL force a change if there is evidence of compromise of the authenticator."

Übung 06: Selbstregistrierung

- Benutzer sollen sich selbst einen Account anlegen können.
- Bei der Registrierung soll die E-Mail-Adresse verifiziert werden.
- Passwörter dürfen weder dem Benutzernamen, noch der E-Mail-Adresse entsprechen, müssen mindestens 10 Zeichen lang sein und mindestens eine Ziffer enthalten.
- Konfiguriere den Realm entsprechend und registriere dich als neuer Benutzer über die Account Console unter `/realms/training/account`.
  Der Sign-in Button befindet sich oben rechts.

Kapitel 5. Benutzer, Rollen & Gruppen

Benutzer anlegen

- In der Admin Console über "Users/Add user" im richtigen Realm
- Eindeutiger Username ist Pflicht
- Required user actions: Aktionen, die der Benutzer beim nächsten Login durchführen muss
- Attributes: Benutzerattribute — welche Attribute zulässig sind, legt das User Profile des Realms fest (siehe unten)
- Credentials: Passwort setzen
- Role mapping & Groups: Rollen und Gruppen zuweisen
- Sessions zeigt alle aktiven Sessions des Benutzers an

User Profile

- Deklaratives Schema für Benutzerattribute je Realm (seit Keycloak 24 Standard)
- Konfiguration unter "Realm settings/User profile"
- Legt fest, welche Attribute existieren, ob sie Pflicht sind, wer sie lesen/schreiben darf und wie sie validiert werden
- Steuert zugleich die Felder im Registrierungs- und Account-Formular
- "Unmanaged attributes" sind per Default deaktiviert — beliebige Zusatzattribute müssen explizit erlaubt bzw. deklariert werden

Rollen

- In der Admin Console über "Realm roles/Create role" im richtigen Realm
- Realm Roles (für alle Clients) vs. Client Roles (nur für einen Client)
- Können ebenfalls beliebige Attribute haben
- Compound Roles enthalten andere Rollen, erzeugen über "Actions/Add associated roles"

Zuweisen von Rollen

- Am Benutzer über "Role mapping/Assign role"
- Als Default-Rolle für alle Benutzer in einem Realm über "Realm roles/default-roles-<realm>/Assign role"
- Als Default-Rolle für Benutzer, die sich über die Selbstregistrierung anmelden über "Realm settings/User registration" auf dem Tab "Default roles"

Übung 07: Benutzer mit Rollen

- In unserem Realm wollen wir über Rollen zwischen Trainern und Schulungsteilnehmern unterscheiden.
- Lege eine Rolle `trainer` und eine Rolle `student` für Trainer bzw. Schulungsteilnehmer an.
- Lege je einen Benutzer für den Trainer Klaus Klöbner und den Schulungsteilnehmer Hermann Wohlgemut an.
- Die Benutzer sollen ein Initialpasswort erhalten, das sie bei der ersten Anmeldung ändern müssen.
- Lege außerdem eine Gruppe `Trainer` an, weise ihr die Rolle `trainer` zu und mache Klaus Klöbner zum Mitglied — so erhält er die Rolle über die Gruppenmitgliedschaft.

Gruppen

- Rollen können einer Gruppe über "Role mapping" zugewiesen werden
- Können ebenfalls beliebige Attribute haben
- Hierarchie von Gruppen über "Child groups" möglich

Typische Nutzung

- Rollen werden zu Gruppen gebündelt
- Benutzer werden zu Gruppen hinzugefügt
- Falls es nur sehr wenige Rollen gibt, kann ggf. auf die Gruppenebene verzichtet werden. Stattdessen können Benutzern dann direkt Rollen zugewiesen werden.

Kapitel 6. LDAP & Active Directory

Anbindung

- Kerberos- und LDAP-Provider können über "Configure/User federation" hinzugefügt werden
- Grundsätzlich wird immer zuerst die lokale Keycloak-Datenbank abgefragt und dann der externe User-Provider
- Fällt ein Provider aus, ist auch eine Authentifizierung mit niedriger priorisierten Providern nicht mehr möglich
- Mapping im Default: username, email, first name und last name, weitere individuelle Mappings konfigurierbar

Synchronisation

- Benutzer aus LDAP können entweder periodisch oder bei Zugriff in die lokale Keycloak-Datenbank übernommen werden
- Passwörter werden nicht synchronisiert, sondern im LDAP-Server geprüft
- Für das Hashing beim Ändern von Passwörtern ist der LDAP-Server zuständig!
- Rückrichtung ebenfalls möglich: In Keycloak angelegte Benutzer können nach LDAP synchronisiert werden ("Sync Registrations")

Übung 08: Anbindung AD

- Konfiguriere für den Realm `training` die Anbindung an ein Active-Directory-Verzeichnis.
- Mit folgenden Einstellungen kannst du eine Verbindung zu einem Active Directory herstellen:
  - Connection URL: `ldap://ad1`
  - Bind DN: `CN=Administrator,CN=Users,DC=ad1,DC=example,DC=com`
  - Bind credentials: `Passw0rd`
- Deaktiviere die Funktion "Sync Registrations", so dass weiterhin auch lokale Benutzer in Keycloak angelegt werden können.
- Überprüfe deine Konfiguration über die Benutzersuche in der Admin Console und über den Login in die Account-Console.
- Tipp: `docker compose exec ad1 ldapsearch` zeigt alle LDAP-Objekte in `ad1` an

Mapping von LDAP-Attributen

- Username LDAP attribute: `sAMAccountName`
- Search scope: One Level oder Subtree
- Über den Tab "Mappers" können LDAP-Attribute auf Keycloak-Attribute abgebildet werden

LDAP Filter

- Beispiel für Benutzer der Gruppe Simpsons:

```
(&(objectClass=person)(sAMAccountName=*)
  (|(memberOf=CN=Simpsons,CN=Users,DC=ad1,DC=example,DC=com)))
```

- `&` verknüpft mit UND
- `|` verknüpft mit ODER

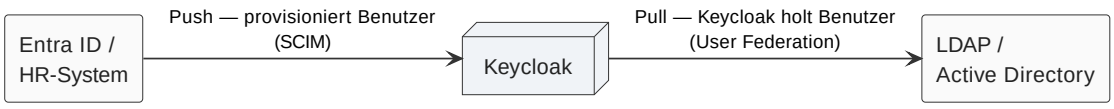
Gruppen & Rollen

- Um Gruppen und Rollen aus LDAP zu übernehmen muss ein `group-ldap-mapper` bzw. `role-ldap-mapper` ergänzt werden.
- Preserve Group Inheritance deaktivieren für flache Gruppenstruktur
- User Groups Retrieve Strategy: `LOAD_GROUPS_BY_MEMBER_ATTRIBUTE`
- Membership LDAP Attribute: `member`
- Membership Attribute Type: `DN`
- Mode: `READ_ONLY`

SCIM

- Gegenstück zur LDAP-Föderation: statt dass Keycloak Benutzer *abholt* (Pull), schiebt ein externes System Benutzer und Gruppen aktiv nach Keycloak (Push)
- Rollenverteilung (wichtig):
  - **SCIM-Server** (Service Provider) = **Keycloak** — stellt den Endpunkt `/realms/<realm>/scim/v2` bereit und legt die Benutzer an
  - **SCIM-Client** (Provisioning-Quelle) = das externe System, das die Daten liefert
- Beispiel Microsoft Entra ID: Benutzer werden primär in **Entra** verwaltet (= SCIM-Client), Entra provisioniert sie automatisch nach **Keycloak** (= SCIM-Server) — inklusive De-Provisioning beim Deaktivieren
- Seit Keycloak 26.8 *supported* und standardmäßig aktiv (Feature `scim-api`)

LDAP (Pull) vs. SCIM (Push)

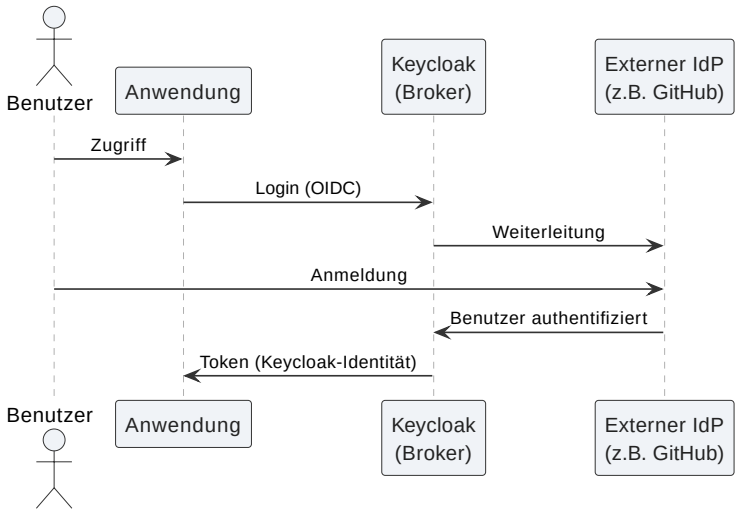


Kapitel 7. Externe Identity Provider

Überblick

- Federation konfigurieren über "Configure/Identity providers"
- Social Login über GitHub, Facebook, Google u.a.
- Integration von IAM-Lösungen von Geschäftspartnern über OpenID Connect v1.0 oder SAML v2.0
- Einbindung cloud-basierter Identity Services
- Seit Keycloak 26.8: überarbeitete Darstellung der IdP-Buttons auf der Login-Seite

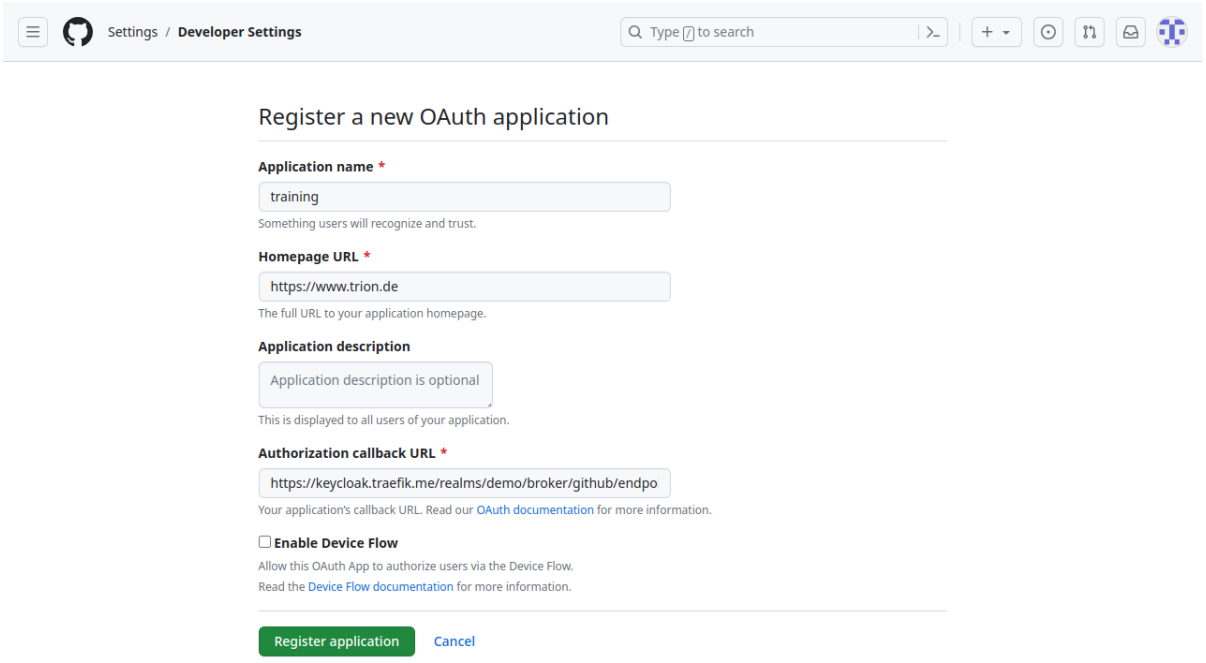
Ablauf: Identity Brokering



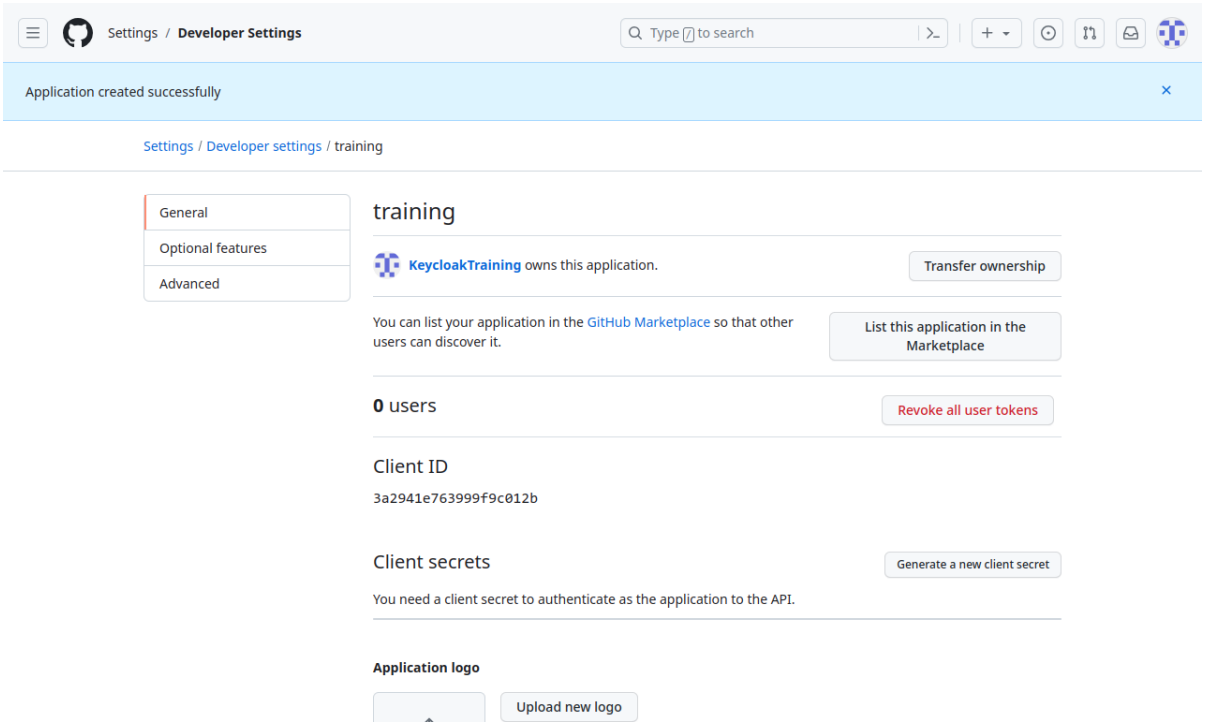
Beispiel: GitHub

- Configure/Identity providers/Add provider/GitHub
- In neuem Tab:
  - GitHub Profil → Settings → Developer Settings → OAuth Apps
  - Register a new application
  - Authorization callback URL aus Feld "Redirect URI" von Keycloak übernehmen
- Client ID und Client Secret von GitHub nach Keycloak übernehmen

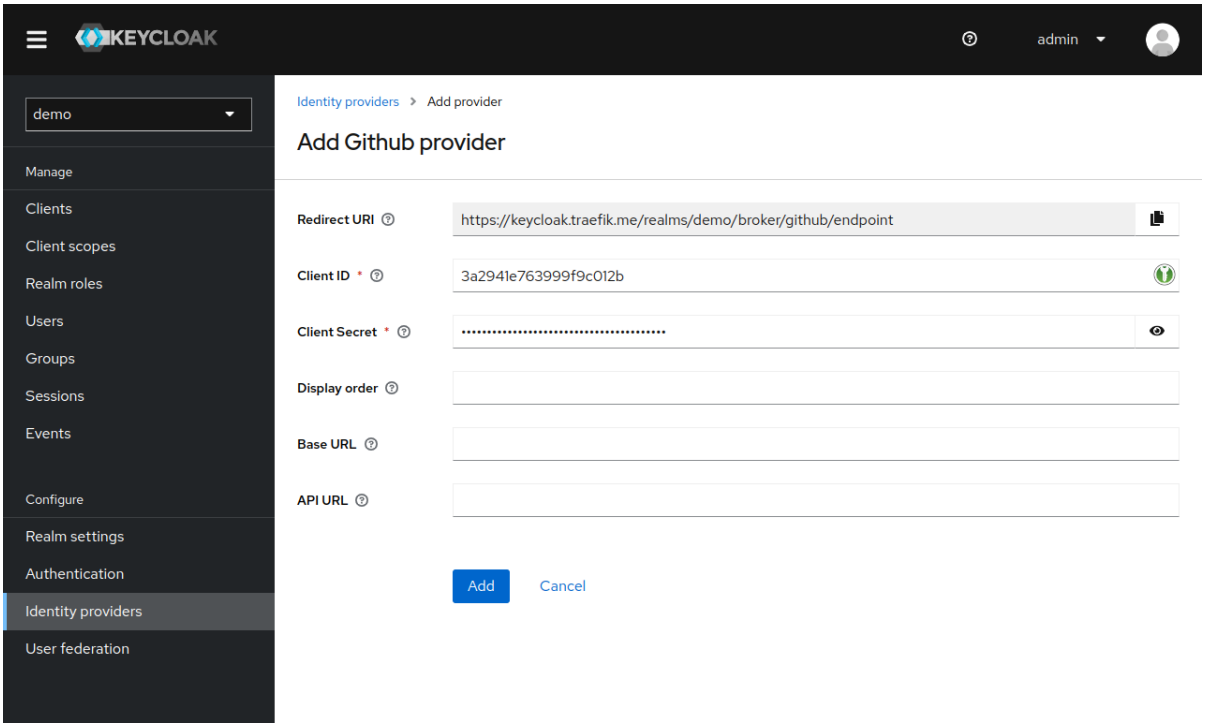
Register Application



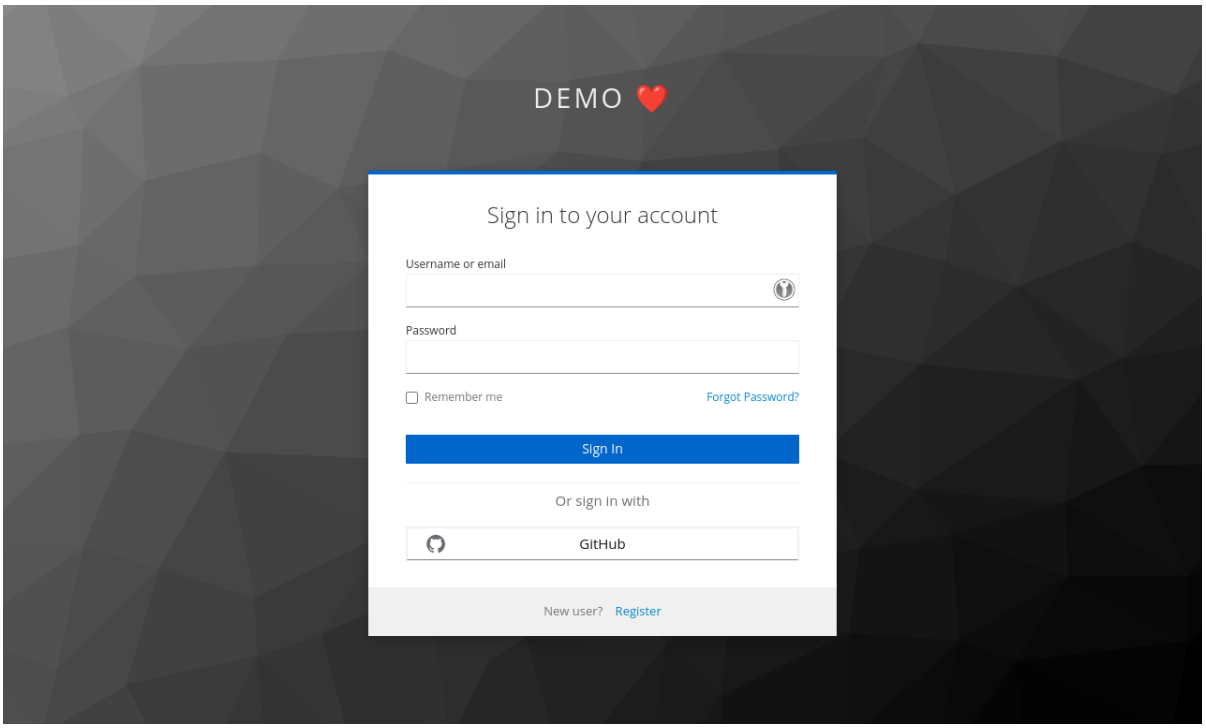
Client Id & Secret



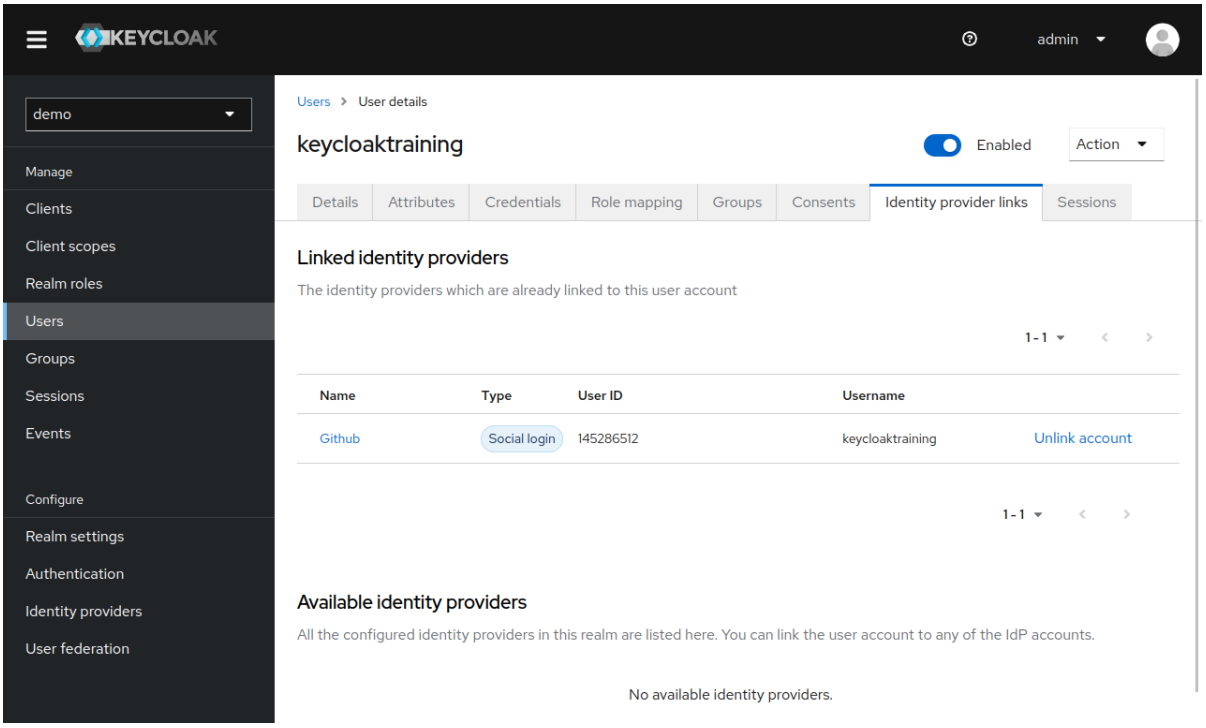
Provider hinzufügen



Sign in with GitHub



Verknüpfung der Accounts



Übung 09: Social Login

- Konfiguriere für den Realm `training` die Funktion "Sign in with GitHub"
- Du kannst entweder über deinen GitHub-Account eine eigene OAuth App anlegen oder folgende Daten verwenden:
  - Client Id: `3a2941e763999f9c012b`
  - Client Secret: `f75db5573ff342b738467757dce6ebd5be092f01`
- Überprüfe deine Konfiguration über die Account Console

Kapitel 8. Organisationen

Was sind Organisationen?

- Neues Konzept in Keycloak 26
- Customer Identity and Access Management (CIAM) mit Fokus auf Business-to-Business (B2B) Use-Cases
- Zusätzliche Strukturierung der Benutzer innerhalb eines Realms

Kernfunktionen von Keycloak Organizations

- Verwaltung der Mitglieder
- Onboarding per Einladungslink oder Identity Brokering
- Identity-First-Login mit organisationsspezifischen Schritten
- Übertragung organisationsspezifischer Claims für autorisierte Zugriffe

Organisationen aktivieren und anlegen

- Aktivieren über Configure/Realm settings/Organizations
- Neuer Menüeintrag Manage/Organizations um Organisationen zu verwalten
- Browser Flow wird automatisch auf "Identity First" umgestellt
- Zuordnung von Domains zu Organisationen um Mitglieder anhand ihrer E-Mail-Adresse zu gruppieren
  - Beispiel:
    - Name: `trion development GmbH`
    - Alias: `trion`
    - Domain: `trion.de`

Mitglieder verwalten

- Realm-User können einer Organisation zugeordnet werden
- Mitglieder können per Invite-Mail eingeladen werden
- Managed vs. Unmanaged User
  - **Managed User** sind Mitglied der Organisation und existieren nur solange die Organisation existiert und aktiv ist
  - **Unmanaged User** können Mitglied der Organisation sein, existieren aber unabhängig von ihr

Identity Provider zuweisen

- Identity Provider werden weiterhin im Realm konfiguriert, können dann aber einer Organisation zugewiesen werden
- Benutzer, die über einen Identity Provider angelegt werden, der einer Organisation zugeordnet ist, werden automatisch Mitglied dieser Organisation

Weiterentwicklung

- Das Feature ist seit Keycloak 26 *supported* und wird laufend ausgebaut
- Bereits verfügbar:
  - **Organization Groups** (seit 26.6): Gruppen innerhalb einer Organisation
  - **Geteilte Identity Provider** (seit 26.8): ein IdP kann mehreren Organisationen zugeordnet werden (many-to-many)
- Noch geplant (siehe Epic Keycloak Organizations):
  - Self-Service für Organisationen
  - Angepasste Templates für Organisationen

Übung 10: Organisation anlegen

- Aktiviere Organizations für den Realm `training`
- Lege eine Organisation `trion` mit der Domain `trion.de` an
- Füge einen bestehenden Realm-Benutzer als Mitglied hinzu
- Lade optional per Invite-Mail ein weiteres Mitglied ein und prüfe die Mail im Mailcatcher

Kapitel 9. Clients

Lernziele

- Public- und Confidential-Clients unterscheiden und anlegen
- Redirect-URIs, Scopes und Mapper korrekt konfigurieren
- Service Accounts, Client Policies und Logout einordnen

Client anlegen

- Anlegen über Manage/Clients/Create client
- Client authentication
  - On: "confidential access type", Client Secret, für Backends
  - Off: "public access type", kein Client Secret, für Frontends (JS)
- Valid redirect URIs, * wird als Wildcard unterstützt
- Valid post logout redirect URIs, falls Logout am IdP genutzt wird
- Web origins für CORS, * um alle Origins zuzulassen
- Unter Credentials kann das Client Secret abgerufen werden

Service Accounts (Client Credentials)

- Für Machine-to-Machine-Zugriff ohne Benutzer (Client Credentials Flow)
- Nur für Clients mit "Client authentication" (confidential)
- Aktivieren über "Capability config/Authentication flow/Service accounts roles"
- Der Client erhält ein eigenes Service-Account-Konto, dem Rollen zugewiesen werden
- Token holen:

```
curl -d grant_type=client_credentials \
  -d client_id=<id> -d client_secret=<secret> \
  https://<HOSTNAME>/realms/<realm>/protocol/openid-connect/token
```

Übung 11: Public Client

- Unter https://www.keycloak.org/app stellt das Keycloak-Projekt eine Test-App für die Anmeldung über Keycloak zur Verfügung.
- Lege einen Client mit Client-Id `test-app` an, konfiguriere dabei insbesondere die "Valid redirect URIs" und die "Web origins".
- Melde dich mit einem Benutzer aus dem Realm bei der Test-App an, bei einer erfolgreichen Anmeldung zeigt sie den Namen des Benutzers

- Bonus: Kopiere das Access Token aus der Test-App und zerlege es auf jwt.io — finde die Claims `sub`, `iss`, `aud`, `exp` sowie deine Rollen

Konfiguration auf Client-Seite

- Discovery
  - `/realms/<realm>/.well-known/openid-configuration`
  - Endpunkt-URLs
  - Link auf Keys/Zertifikate
  - Unterstützte Protokolle und Algorithmen
- Client Id und Secret
- ggfs: Scopes, Username Attribute
- Konkrete Konfiguration stark abhängig vom verwendeten Framework/Plattform

Client Scopes & Mapper

- Client Scopes bündeln wiederverwendbare Protocol Mapper und Rollen-Scopes
- Default- vs. Optional-Scopes: Default-Scopes sind immer im Token, optionale nur auf Anfrage (`scope`-Parameter)
- Protocol Mapper bestimmen, welche Claims in Access Token, ID Token und UserInfo landen (z.B. Benutzerattribute, Rollen, Gruppen)
- Konfiguration je Client unter "Clients/<client>/Client scopes" bzw. global unter "Client scopes"
- Nützlich, um ausgestellte Claims und die Token-Größe gezielt zu steuern

Autorisierung in der Anwendung

- Keycloak übernimmt die **Authentifizierung** (wer ist der Benutzer?) — die **Autorisierung** (was darf er?) entscheidet meist die Anwendung
- Rollen und Gruppen werden als Claims ins Token gemappt (z.B. `realm_access.roles`, `groups`)
- Die Anwendung liest diese Claims und trifft darauf ihre Zugriffsentscheidungen (z.B. Spring Security: `hasRole(...)`)
- Für feingranulare, ressourcenbasierte Autorisierung bietet Keycloak zusätzlich **Authorization Services** (UMA 2.0)

Logout

- Ohne Logout beim IdP wird der User beim nächsten Login-Versuch typischerweise direkt wieder angemeldet, um dies zu verhindern, sollte die Anwendung den Logout an den IdP propagieren.
- OIDC Spezifikationen:
  - Session Management: Browser-basiert, regelmäßiges Polling auf Keycloak, Logout in der

Anwendung sobald Session terminiert wurde

- RP-Initiated Logout: Browser-basiert, Logout wird von Anwendung initiiert
- Front-Channel Logout: iframe-basiert, propagiert Logout von Keycloak zur Anwendung
- Back-Channel Logout: Keycloak propagiert Logout per HTTP POST zur Anwendung

Client Policies & FAPI

- Zentrale, wiederverwendbare Sicherheitsvorgaben für Clients — statt jeden Client einzeln abzusichern
- Zwei Bausteine:
  - **Profile** = Satz von **Executors**, die Anforderungen erzwingen (z.B. PKCE-Pflicht, nur confidential Clients, PAR, Secret-Rotation)
  - **Policy** = **Conditions** (auf welche Clients das Profil wirkt) + Zuordnung eines Profils
- Keycloak bringt fertige Profile mit, u.a. **FAPI 1.0 Baseline/Advanced**, **FAPI 2.0** (Financial-grade API) und **OAuth 2.1**
- Konfiguration unter "Realm settings/Client policies"

FAPI-Bausteine: PAR & Holder-of-Key

- **PAR** — Pushed Authorization Requests (RFC 9126)
  - Client sendet die Request-Parameter vorab per Back-Channel und erhält eine kurzlebige `request_uri`
  - Der Browser-Redirect trägt nur noch `client_id` + `request_uri` → nichts Manipulierbares in der URL
- **Holder-of-Key** — Sender-Constrained Tokens
  - Das Token ist an einen Schlüssel des Clients gebunden; ein gestohlenes Token allein ist wertlos
  - **mTLS** (RFC 8705): Bindung an das Client-TLS-Zertifikat
  - **DPoP** (RFC 9449): Bindung per signiertem Proof je Request — ohne mTLS-Infrastruktur
- Durchsetzung typischerweise über die FAPI-Client-Policies

PAR & Holder-of-Key: Trade-offs

- Lokale Token-Validierung bleibt erhalten — der Resource Server braucht weiterhin **keine** Rückfrage an den Authorization Server (JWKS gecacht, Prüfung lokal)
- **mTLS**: erfordert Client-Zertifikats-PKI; der Resource Server bzw. der vorgelagerte Proxy muss das Client-Zertifikat sehen — TLS-Terminierung/Loadbalancer müssen es durchreichen (vgl. mTLS-Fallstricke bei Traefik)
- **DPoP**: Client signiert pro Request einen Proof; gegen Replay braucht der Resource Server `jti` -/Nonce-Prüfung (ggf. geteilter Zustand) — oder die geschützten Operationen sind idempotent,

dann bleibt ein Replay folgenlos

- **PAR**: zusätzlicher Round-Trip vor dem Redirect, kurzzeitiger Server-Zustand, Client-/Bibliotheks-Support nötig

Grenze: Aufrufketten

- Sender-constrained Tokens (DPoP **und** mTLS) lassen sich **nicht** weiterreichen (Service1 → Service2 → Service3) — der nächste Dienst besitzt den Bindungsschlüssel nicht
- Lösung: **Token Exchange** (RFC 8693) — jeder Hop tauscht das eingehende Token gegen ein eigenes (mit eigener Bindung und passender Audience/Scope)
- Gilt auch als bessere Praxis: blindes Weiterreichen desselben Bearer-Tokens durch die ganze Kette ist ohnehin ein Anti-Pattern (zu breite Audience, großer Blast-Radius)
- Fazit: mehr Komplexität in Client, Resource Server und Infrastruktur — nur einsetzen, wo das Schutzniveau es rechtfertigt

Client Secret Rotation

- Seit Keycloak 26.8 *supported* und standardmäßig aktiv (nur für Clients mit "Client authentication")
- Keycloak hält pro Client bis zu **zwei gleichzeitig gültige Secrets** → Rotation ohne Downtime
- Konfiguration nicht am Client selbst, sondern über **Client Policies**:
  - Unter "Realm settings/Client policies/Profiles" ein Profil mit dem Executor `secret-rotation` anlegen
  - Über eine Policy mit passenden Conditions den betroffenen Clients zuordnen
- Parameter (jeweils in Sekunden):
  - Secret expiration: Lebensdauer des neuen Secrets
  - Rotated Secret expiration: Restgültigkeit des alten Secrets (muss kleiner sein; `0` = sofort entfernen)
  - Remain Expiration Time: Restlaufzeit, ab der ein Update eines (dynamischen) Clients eine Rotation auslöst

Client Registration Service

- Applikationen oder Services können selbst den Client in Keycloak anlegen
- Authentifizierung über Bearer-Token oder "Initial access token" (erzeugen unter Clients, Tab "Initial access token")
- Client Registration Policies definieren Rahmenbedingungen für authentifizierte und unauthentifizierte Requests gegen den Client Registration Service
- Details in Securing Apps

Kapitel 10. Mehrfaktorauthentifizierung

Lernziele

- TOTP/HOTP und WebAuthn/Passkeys einrichten
- Recovery Codes als Backup-Faktor einordnen
- Authentication Flows anpassen

MFA

- Keycloak unterstützt als zweiten Faktor neben dem Passwort out of the box TOTP/HOTP über Mobile Apps und WebAuthn.
- Weitere Verfahren können über ein Service Provider Interface (SPI) entwickelt und eingebunden werden.

Recovery Codes

- Einmal-Codes als Backup, falls der primäre zweite Faktor nicht verfügbar ist
- Seit Keycloak 26 *supported* (Feature `recovery-codes`)
- Einrichten über die Required Action "Generate Recovery Codes" oder durch Benutzer in der Account Console
- Jeder Code ist nur einmal verwendbar; die Liste muss sicher aufbewahrt werden

TOTP/HOTP

- Konfiguration über "Authentication/Policies/OTP Policy"
  - Time Based (TOTP)
  - Counter Based (HOTP)
- Einrichten über Admin Console (Users/Required user actions/Configure OTP) oder durch Benutzer in der Account Console
- Mobile Apps: FreeOTP, Google Authenticator, Microsoft Authenticator
- Zum Ausprobieren: https://totp.danhersam.com/

Übung 12: TOTP

- Aktiviere für einen Benutzer TOTP über die Required Action "Configure TOTP"
- Melde dich mit diesem Benutzer in der Account Console an und registriere einen TOTP Generator
- Melde dich ab und wieder an und prüfe, dass die Anmeldung nur akzeptiert wird, wenn ein gültiges One-Time-Password eingegeben wird.

WebAuthn

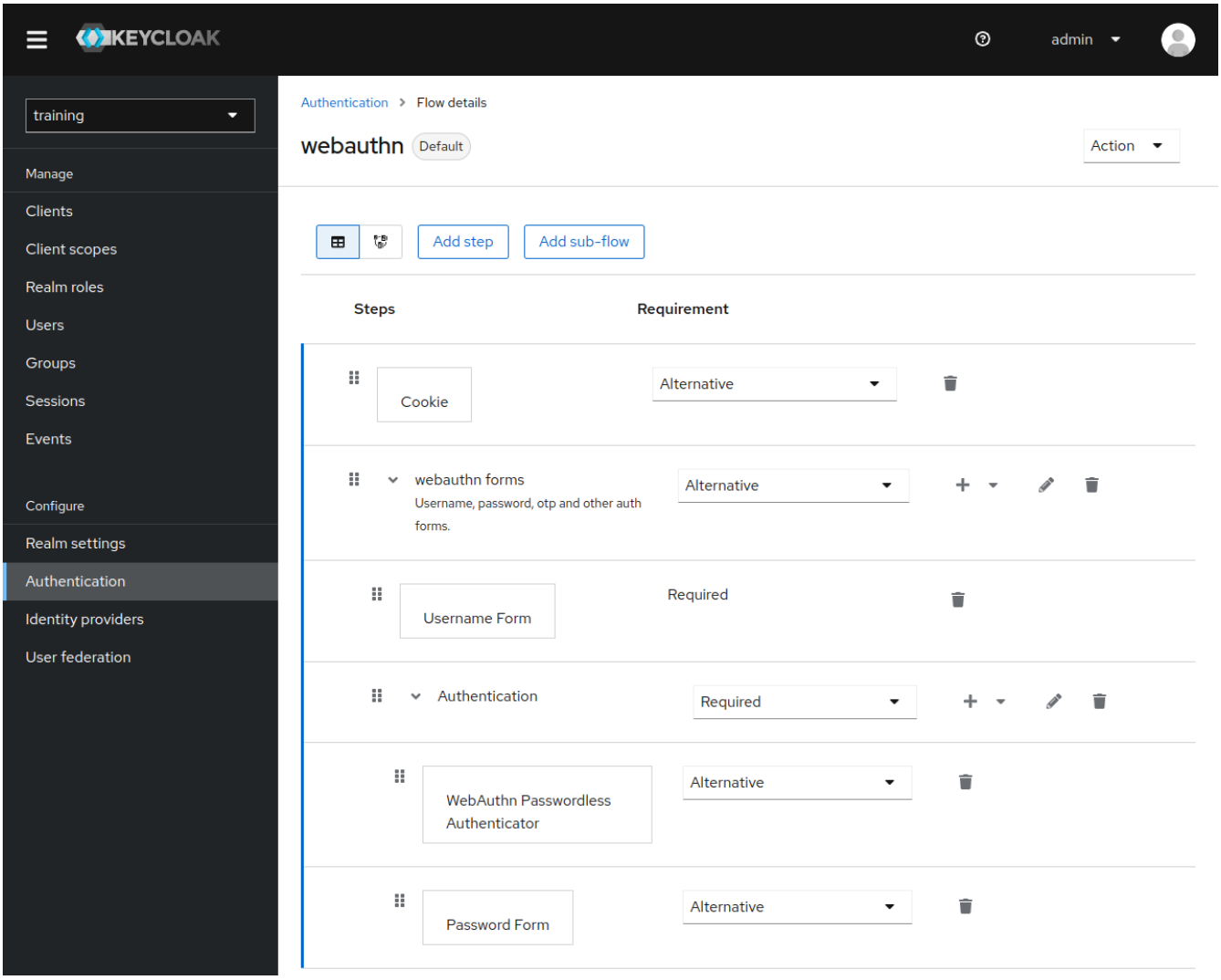
- W3C Standard
- Einrichten über Admin Console (Users/Required user actions/Webauthn Register bzw. Webauthn Register Passwordless)
- Unterstützt Hardware Tokens (z.B. YubiKey 5) und Passkeys
- Chrome Developer Tools unterstützen virtuelle WebAuthn Authenticator: More Options > More tools > WebAuthn.
- Verhalten konfigurierbar über "Authentication/Policies" (WebAuthn Policy bzw. WebAuthn Passwordless Policy)
- WebAuthn als 2. Faktor ist im Default-Browser-Flow bereits enthalten (unter "Browser - Conditional 2FA", standardmäßig `DISABLED`) — zum Aktivieren nur auf `ALTERNATIVE` stellen; **Passwordless** erfordert weiterhin einen eigenen Flow (siehe Übung 13)



WebAuthn und Passkeys sind inzwischen fester, *supported* Bestandteil von Keycloak (kein Preview-Feature mehr). Ältere Anleitungen, die dafür Feature-Flags setzen, sind überholt.

Authentication Flows

- Werden über "Configure/Authentication/Flows" verwaltet
- Ein Flow kann über "Action/Duplicate" dupliziert werden (Dropdown oben rechts)
- Über "Action/Bind flow" kann der Flow aktiviert werden



Übung 13: WebAuthn

- Erstelle und aktiviere einen Authentication Flow, der WebAuthn Passwordless unterstützt
- Lege einen neuen Benutzer mit einer Required Action "WebAuthn Register Passwordless" an
- Nutze den Chrome Browser, aktiviere den virtuellen WebAuthn Authenticator und melde dich an der Account Console an
- Registriere deinen WebAuthn Authenticator und nutze ihn dann zur Authentifizierung

Individuelle Lösungen

- Eigene Implementierungen sind über das Authentication SPI möglich
  - erlaubt u.a. eigene Authenticator, Required Actions zu entwickeln und einzubinden
- Dokumentation im Server Developer Guide
- Beispiele u.a. in keycloak/examples

Kapitel 11. Themes

Organisation von Themes

- Themes werden unterhalb von `themes` in einem Verzeichnis mit dem Schema `<theme name>/<theme type>` abgelegt
  - `themes/demo/login` für das Login-Theme mit dem Namen `demo`
  - Wenn Docker-Image verwendet wird: `/opt/keycloak/themes/`
- Alternativ: Verteilung als JAR und Installation über Verzeichnis `providers`
- Standard-Themes, die mit Keycloak ausgeliefert werden, liegen in `/opt/keycloak/lib/lib/main/org.keycloak.keycloak-themes-<version>.jar` und bei GitHub
- Theme für die Account Console liegt in `/opt/keycloak/lib/lib/main/org.keycloak.keycloak-account-ui-<version>.jar`

Theme Types

- account: Account Management Console
- admin: Admin Console
- email: Von Keycloak versendete E-Mails
- login: Login Formulare inkl. Passwort vergessen, Registrierung, etc.
- welcome: Willkommensseite

Woraus besteht ein Theme?

- HTML (Freemarker Templates)
- Bilder in `resources/img`
- Message Bundles in `messages`
- CSS in `resources/css`
- Javascript in `resources/js`
- Theme Properties in `theme.properties`

Entwicklung von Themes

- Während der Entwicklung helfen die Startoptionen
  - `--spi-theme-static-max-age=-1`
  - `--spi-theme-cache-themes=false`
  - `--spi-theme-cache-templates=false`
- Deaktiviert Caching und erlaubt Testen von Änderungen über einfachen Browser-Reload

theme.properties

- Properties-Datei in `<THEME TYPE>/theme.properties` innerhalb des `themes`-Verzeichnisses.
  - `parent`: Eltern-Theme, von dem dieses Theme abgeleitet wird
  - `import`: Importiert Ressourcen von einem anderen Theme
  - `styles`: Leerzeichenseparierte Liste der einzubindenden CSS-Dateien
    Hinweis: auch CSS-Dateien aus dem Parent müssen explizit mit aufgeführt werden
  - `locales`: Kommaseparierte Liste der unterstützten Locales

Beispiel: Custom CSS für Login

- `themes/demo/login/theme.properties`

```
parent=keycloak.v2

styles=css/styles.css css/demo.css
```

- `themes/demo/login/resources/css/demo.css`

```
#kc-header {
  color: #ff0000 !important;
}

#kc-header-wrapper {
  font-size: 99px;
}
```

Übung 14: Theme

- Erstelle ein eigenes abgeleitetes Login-Theme bei dem du das Hintergrundbild der Login-Seite austauschst.
- Folgendes CSS-Snippet kann als Inspiration dienen:

```
.login-pf body {
    background: url("https://images.unsplash.com/photo-1693773024865-d2623880567d")
      no-repeat center center fixed;
    background-size: cover;
    height: 100%;
}
```

Tipps & Tools

- UI Customization Guides
- Für Anpassungen an den Freemarker Templates: Template Author's Guide
- Unterliegende Datenmodelle sind nicht explizit dokumentiert, oft hilft der Quellcode, z. B. FreeMarkerLoginFormsProvider für Login Themes
- Keycloakify generiert React Templates für Keycloak

Kapitel 12. Plugins

Erweiterung von Keycloak

- Bestehende Funktionalität kann geändert bzw. ersetzt oder erweitert werden
- Dokumentation im Server Developer Guide

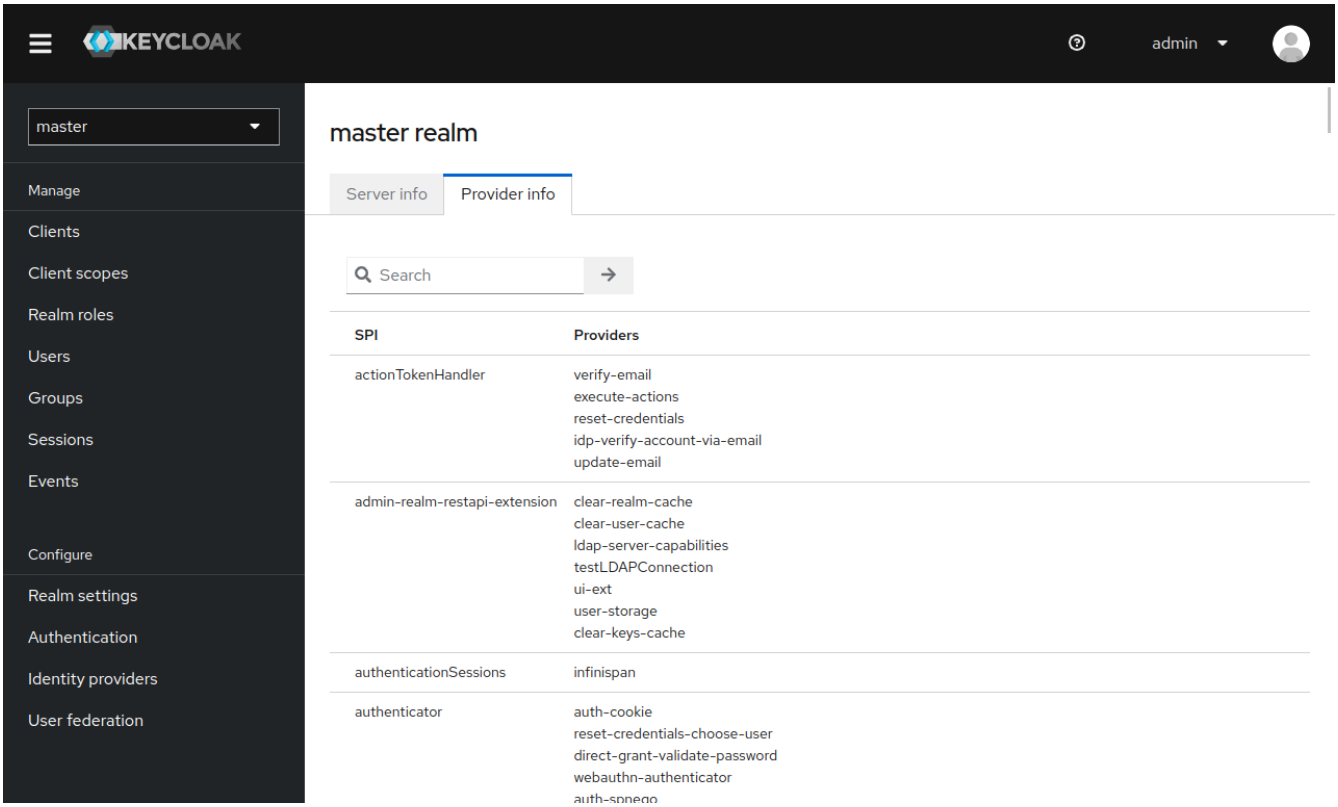
Tipps zur Entwicklung

- Keycloak über docker compose im Development Mode nutzen
- Nützliche Umgebungsvariablen setzen:
  - `DEBUG: 'true'`
  - `DEBUG_PORT: '*:8787'`
  - `KC_LOG_LEVEL: INFO` oder `KC_LOG_LEVEL: DEBUG`
- Maven `target`-Verzeichnis nach `/opt/keycloak/providers` mounten
- Bei Änderungen: `mvn verify` und Keycloak neu starten, kein hot reloading

Service Provider Interfaces

- Die verfügbaren Service Provider Interfaces (SPI) können in der Admin Console abgerufen werden (Startseite, Tab "Provider Info")
- Es wird unterschieden zwischen
  - *Single-implementation provider types*: Es gibt genau eine Komponente, die das Interface implementiert, z.B. `HostnameProvider`
  - *Multiple implementation provider types*: Mehrere Komponenten implementieren das Interface, z.B. `Authenticator`

Provider Info



Authentication SPI

- Erlaubt es neben den im Standard enthaltenen Authentifizierungsverfahren (Passwort, Kerberos, OTP, WebAuthn, etc.) weitere eigene Verfahren zu ergänzen.
- Das Registrierungsformular für neue Benutzer kann angepasst oder ersetzt werden.
- Required Actions, das heißt Aktionen, die der Benutzer nach der Anmeldung einmalig ausführen muss, können ergänzt werden.

Action Token Handler SPI

- Action Tokens sind spezielle Json Web Tokens (JWT), die es einem Benutzer erlauben eine bestimmte Aktion auszuführen.
- Im Standard werden folgende Aktionen unterstützt:
  - Passwort zurücksetzen
  - E-Mail-Adresse bestätigen
  - "Required Action" ausführen, z.B. Nutzungsbedingungen akzeptieren
  - Verknüpfung eines Accounts mit einem Account bei externen IdP bestätigen

Event Listener SPI

- Implementieren von `EventListenerProvider` und `EventListenerProviderFactory`
- Erlaubt es auf *User-Events* (Login, Registrieren, ...) und *Admin-Events* (Client erzeugen/ändern, Realm konfigurieren, ...) zu reagieren

```
public interface EventListenerProvider extends Provider {

    void onEvent(Event event);

    void onEvent(AdminEvent event, boolean includeRepresentation);
}
```

Protocol Mapper SPI

- Steuert, welche Informationen (Benutzerattribute, Rollen, Gruppen, ...) in die ausgestellten Tokens bzw. das UserInfo gemappt werden
- Keycloak liefert bereits zahlreiche Standard-Mapper mit (z.B. `oidc-usermodel-attribute-mapper`, `oidc-usermodel-client-role-mapper`)
- Über die SPI lassen sich eigene Mapper ergänzen, z.B. um berechnete oder aus Fremdsystemen angereicherte Claims auszustellen
- Implementieren von `ProtocolMapper` bzw. der passenden OIDC-/SAML-Basisklasse

Vault SPI

- Erlaubt es Keycloak, Secrets (z.B. das SMTP- oder LDAP-Bind-Password) aus externen Vaults zu lesen, wo sie sicher gespeichert werden
- Im Standard wird ein Vault auf Basis von Textdateien (`file`), vor allem zur Nutzung mit Kubernetes Secrets, und auf Basis von Java Keystores (`keystore`) unterstützt
- Konfiguration ist im Guide Using a vault beschrieben

Erweiterungen des Servers

- Es können REST-Endpunkte ergänzt werden (über `RealmResourceProviderFactory` and `RealmResourceProvider`)
- Es können neue SPIs definiert werden (über `org.keycloak.provider.Spi` und `ProviderFactory` and `Provider`)
- Es können neue JPA-Entities definiert werden, um so das Datenmodell zu erweitern (über `JpaEntityProviderFactory` und `JpaEntityProvider`)

Kapitel 13. REST API für Administration

Admin CLI

- Authentifizieren über `kcadm.sh config credentials`

```
docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  config credentials --server http://localhost:8080 \
  --realm master --user admin --password admin
```

- Benutzer abfragen über `kcadm.sh get users`

```
docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  get users -r training
```

- Benutzer anlegen und Passwort setzen

```
docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  create users -r training -s username=testuser -s enabled=true

docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  set-password -r training --username testuser --new-password testpassword
```

REST API

- Direkte Nutzung der REST API ermöglicht die Integration in eigene Anwendungen oder Skripte
- Basis URL: `{base url}/admin/realms` z.B. `https://keycloak.example.com/admin/realms`
- Dokumentation der Ressourcen unter https://www.keycloak.org/docs-api/latest/rest-api/
- Da sowohl `kcadm.sh` als auch die Admin Console diese API nutzen, lässt sich darüber grundsätzlich jeder Use-Case realisieren, der über die Kommandozeile oder die Admin Console zur Verfügung steht

Praxis: Benutzerliste für eine Anwendung

- Anwendungsfall: eine App (Keycloak-Client) braucht eine Benutzerliste, z.B. ein Dropdown zum Zuweisen von Tasks an Mitglieder einer Gruppe
- **Nicht** über OIDC-Tokens lösbar — die beschreiben nur den angemeldeten Benutzer. Dafür die **Admin REST API** nutzen
- Die App authentifiziert sich als **Service Account** (Client Credentials) mit der Rolle `view-users` (vom Client `realm-management`)
- Mitglieder einer Gruppe abfragen:

```
GET /admin/realms/<realm>/groups/<group-id>/members?briefRepresentation=true
Authorization: Bearer <service-account-token>
```

- `first`/`max` für Pagination beachten; Ergebnisse in der App zwischenspeichern statt bei jedem Seitenaufruf neu abzufragen

Export und Import

- Export über Kommandozeile:

```
docker compose exec keycloak /opt/keycloak/bin/kc.sh export --dir /tmp/export
```

- Beim Import darf die Keycloak-Instanz nicht laufen!
- Import entweder über `kc.sh import` analog zu Export oder beim Startup über die Kommandozeilenoption `--import-realm` aus dem Verzeichnis `/opt/keycloak/data/import`
- Gut geeignet zum Aufsetzen von Regressionstests mit definiertem Satz von Testdaten
- Details unter Importing and Exporting Realms
- Konfiguration als JSON/YAML verwalten: adorsys/keycloak-config-cli

Übung 15: Admin REST API

- Authentifiziere dich mit `kcadm.sh config credentials` gegen den `master`-Realm
- Lege im Realm `training` über `kcadm.sh create users` einen neuen Benutzer an und setze ein Passwort
- Prüfe über `kcadm.sh get users -r training`, dass der Benutzer angelegt wurde
- Kontrolliere das Ergebnis zusätzlich in der Admin Console

Kapitel 14. Anpassungen für Produktivbetrieb

Lernziele

- Produktionsreif konfigurieren (DB, Backup, Monitoring, Logging)
- Keycloak absichern (Hardening-Checkliste)
- Häufige Fehler erkennen und beheben

Tipps

- Nicht die In-Memory-Datenbank aus dem Default verwenden
  - Stattdessen typischerweise MariaDB oder PostgreSQL
- Backup einrichten und regelmäßig Restore testen
- Testumgebung bereitstellen, um Upgrades vor Inbetriebnahme ausgiebig testen zu können
- Prüfen, ob High Availability (Cluster-Betrieb) erforderlich ist
- Falls mehr als geringfügige Last erwartet wird: Lasttests durchführen
- Betrieb hinter Reverse Proxy (z.B. TLS-Terminierung, zusätzliche Absicherung Admin-Console, WAF)

Backup & Restore

- Entscheidend ist die **Datenbank** — sie enthält Realms, Benutzer, Clients und Konfiguration
- Konsistentes Backup mit den Datenbank-Werkzeugen, z.B. PostgreSQL:

```
docker compose exec db pg_dump -U keycloak keycloak > backup.sql
```

- Restore in eine leere Datenbank:

```
cat backup.sql | docker compose exec -T db psql -U keycloak keycloak
```

- Restore regelmäßig testen; der Realm-Export (JSON) ist eine menschenlesbare Ergänzung, aber kein Ersatz für ein DB-Backup

Health-Endpunkte

- Keycloak stellt Health-Endpunkte auf Port 9000 bereit (aktiviert über `KC_HEALTH_ENABLED=true`)
- Verfügbare Endpunkte:
  - `/health`: Gesamtstatus (`UP` oder `DOWN`)
  - `/health/ready`: Bereit für Anfragen (Datenbank erreichbar, Startup abgeschlossen)

    - `/health/live`: Prozess läuft (für Kubernetes Liveness Probe)
    - `/health/started`: Initialer Startup abgeschlossen (für Kubernetes Startup Probe)
- Endpunkte liefern JSON und sind nicht authentifizierungspflichtig
- Sollten **nicht** öffentlich exponiert werden — im Reverse Proxy blockieren oder nur intern erreichbar lassen

```
# Beispiel: Kubernetes Readiness Probe
readinessProbe:
  httpGet:
    path: /health/ready
    port: 9000
  initialDelaySeconds: 30
  periodSeconds: 10
```

Metriken und Monitoring

- Keycloak stellt Metriken über Micrometer bereit
- Metriken können über die Umgebungsvariable `KC_METRICS_ENABLED=true` aktiviert werden und können dann per HTTP unter `/metrics` auf Port 9000 abgerufen werden
- Dashboards und Alerts über Prometheus und Grafana
- Überwachung der Log-Ausgaben (z.B. Loki oder Elasticsearch)

Logging

- Log-Level wird über `KC_LOG_LEVEL` gesteuert (Standard: `INFO`)
- Format über `KC_LOG_CONSOLE_FORMAT` anpassbar: Standard ist strukturiertes Textformat, alternativ JSON
- JSON-Logging aktivieren: `KC_LOG_CONSOLE_OUTPUT=json`
    - Vereinfacht maschinelle Auswertung in Log-Aggregatoren (Loki, Elasticsearch)

```
KC_LOG_LEVEL: INFO
KC_LOG_CONSOLE_OUTPUT: json
```

- Für gezieltes Debugging einzelner Subsysteme kann der Log-Level pro Kategorie gesetzt werden:

```
KC_LOG_LEVEL: INFO,org.keycloak.authentication:DEBUG
```



`DEBUG`-Level auf gesamtem Keycloak erzeugt sehr viele Logs und enthält ggf. sensible Daten — nur kurzfristig in Produktivumgebungen einsetzen.

Feature Flags

- Optionale Features über `KC_FEATURES` aktivieren bzw. über `KC_FEATURES_DISABLED` deaktivieren
- Keycloak unterscheidet u.a. folgende Reifegrade:
  - **Supported**: stabil, teils standardmäßig aktiv (z.B. `organizations`, `persistent-user-sessions`, `update-email`, `recovery-codes`)
  - **Preview**: nutzbar, aber (noch) nicht für den Produktivbetrieb empfohlen (z.B. `token-exchange`)
  - **Deprecated**: auslaufend (z.B. `multi-site` zugunsten von `stateless`)
- Viele früher als Preview markierte Features (z.B. Organizations, persistente Sessions, Update-Email, Recovery-Codes) sind inzwischen **GA und standardmäßig aktiv**
- Details unter Enabling and disabling features

Brute-Force-Schutz

- Aktivierung je Realm über "Realm settings/Security defenses/Brute force detection"
- Verschiedene Strategien: dauerhaftes oder temporäres Sperren des Accounts, nur Sperren der Client-Verbindung
- Parameter u.a. für maximale Fehlversuche, Wartezeit und Backoff
- Seit Keycloak 26.8 werden die fehlgeschlagenen Logins per Default in der **Datenbank** gespeichert (`login-failures:v2`)
  - Sperren überstehen damit Neustarts und Cluster-Wechsel einzelner Nodes

Security-Hardening-Checkliste

- TLS erzwingen (Require SSL = "all requests"), HSTS am Reverse Proxy
- Öffentliche Clients: PKCE verpflichtend; Backends als confidential mit Secret
- Redirect-URIs strikt halten (keine offenen Wildcards), Web Origins eng fassen
- Kurze Access-Token-Lebensdauer, Refresh-Token-Rotation
- Brute-Force-Schutz aktivieren, sinnvolle Password Policy setzen
- Admin-Console zusätzlich absichern (nur intern bzw. hinter WAF)
- Health- und Metrics-Endpunkte (Port 9000) nicht öffentlich exponieren
- Secrets aus Vault/Umgebung statt Klartext; regelmäßig Updates einspielen

Häufige Fehler & Troubleshooting

- **Invalid redirect URI**: Redirect-URI stimmt nicht exakt (Schema/Port/Pfad) → Client-Einstellungen prüfen
- **Timeout / Endlos-Redirect beim Login**: `KC_HOSTNAME` falsch (http vs. https, Port) → korrekt

setzen und neu starten

- **CORS-Fehler im Browser**: Web Origins am Client fehlen oder sind zu eng
- **401 trotz gültigem Login**: falsche Token-Audience (`aud`) → passenden Client-Scope/Mapper setzen
- **Signatur-/exp-Fehler am Resource Server**: Uhren nicht synchron (Clock Skew) oder veraltete JWKS gecacht
- Gezielt mehr Logs: `KC_LOG_LEVEL: INFO,org.keycloak.events:DEBUG`

Kubernetes

- Keycloak Operator
- Unterstützt Deployment und Konfiguration von
  - Datenbank
  - Hostname
  - TLS-Zertifikate und private Schlüssel
- Keycloak Realm Import

Kapitel 15. Anhang: Betrieb im Cluster

Keycloak Cluster

- Keycloak kann im Cluster betrieben werden
- Höhere Verfügbarkeit
- Höherer Durchsatz

Caches

- Keycloak nutzt neben der Datenbank als (verteilten) Cache Infinispan (high-performance, distributable in-memory data grid)
  - Lokale Caches für Realms, User, Authorization und externe öffentliche Schlüssel
  - Replicated Cache (auf alle Nodes repliziert) für Invalidierungsnachrichten
  - Distributed Caches u.a. für Authentication Sessions, fehlgeschlagene Logins und Action Tokens
- Default Konfiguration für Infinispan in `/opt/keycloak/conf/cache-ispn.xml`



Seit Keycloak 26 ist `persistent-user-sessions` standardmäßig aktiv — Benutzer- und Client-Sessions werden in der **Datenbank** gehalten (und nur zusätzlich gecacht) und überstehen damit Neustarts. Die reine In-Memory-Session-Haltung früherer Versionen ist überholt.

Konfiguration der Caches

- Verschiedene Optionen
  - Eigene Infinispan Konfigurationsdatei über `KC_CACHE_CONFIG_FILE` setzen
  - Externen Infinispan-Server verwenden (`KC_CACHE_REMOTE_HOST`, `_PORT`, `_USERNAME` und `_PASSWORD`)
  - Default Konfiguration verwenden und Transport Stack festlegen (`KC_CACHE_STACK`): `tcp`, `udp`, `kubernetes` und Cloud-Provider spezifische Stacks für AWS, Google und Azure
- Guide: Configuring distributed caches



Kommunikation der Caches untereinander sollte abgesichert werden, wenn das Netzwerk nicht sicher ist. Siehe Securing cache communication.

Kubernetes Transport Stack

- Nutzt zur Discovery `DNS_PING`, d.h. ermittelt Cluster-Nodes aus `A`- oder `SRV`-Records
- Nutzt TCP als Transportprotokoll
- Gut geeignet für Betrieb in Kubernetes, Docker Swarm aber auch in anderen Umgebungen,

wenn Zugriff auf DNS besteht

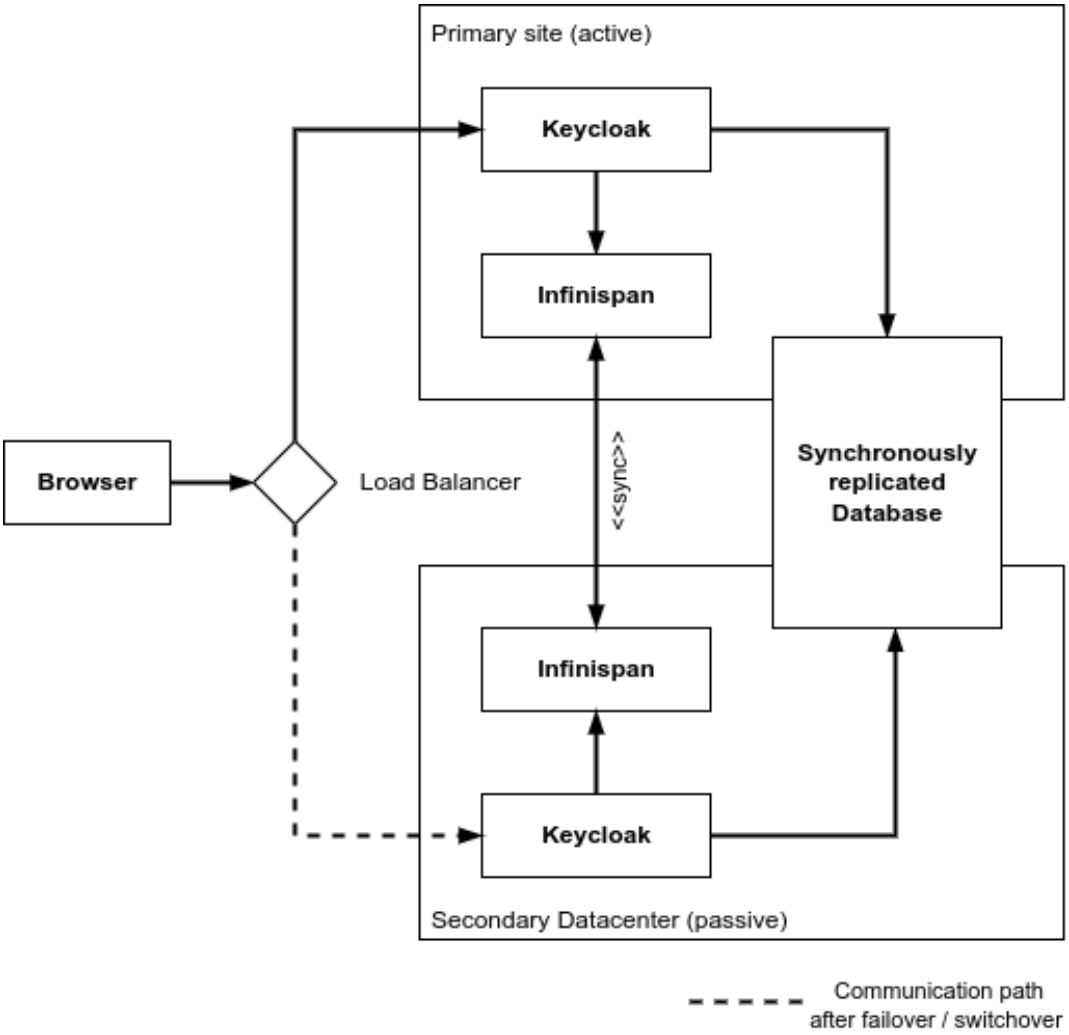
- `KC_CACHE_STACK: kubernetes`
- Zusätzlich muss der abzufragende DNS-Record über `-Djgroups.dns.query=` gesetzt werden

Installation in Kubernetes

- Kubernetes Operator für Keycloak
- Einfaches Verwalten von Keycloak Clustern
- Bereitgestellte CRDs:
  - `Keycloak` für Keycloak Cluster
  - `KeycloakRealmImport`
- Admin Passwort: `kubectl get secret example-kc-initial-admin -o jsonpath='{.data.password}' | base64 --decode`

Multi-Site Deployments

- Synchrone Replikation in zweites Rechenzentrum
- In jedem RZ können mehrere Keycloak-Nodes laufen
- Externer Infinispan Cache
- Synchron replizierte Datenbank
- Vorgeschalteter Loadbalancer schwenkt bei Ausfall des primären RZ
- Guide: Multi-site deployments





Diese klassische Multi-Site-Architektur (Feature `multi-site`, "v1") gilt seit Keycloak 26.8 als deprecated und wird in einer künftigen Major-Version entfernt. Empfohlener Nachfolger ist Multi-Cluster v2 (`stateless`).

Multi-Cluster v2 (stateless)

- Seit Keycloak 26.8 *supported*, löst das bisherige `multi-site` (v1) ab
- Aktivieren über das Feature `stateless`:
  - `KC_FEATURES: stateless` bzw. `--features=stateless`
- **Kein externes Infinispan-Cluster mehr nötig** — die Deployment-Architektur wird deutlich einfacher
- Sessions werden in der **Datenbank** gehalten statt im Speicher oder in einem externen Cache
- Ein externer Loadbalancer verteilt den Traffic über die Cluster
- Trade-off: höhere Last auf der Datenbank (ca. doppelte CPU-Last und Write-IOPS)
- Guide: Multi-site deployments

Kapitel 16. Anhang: Cheat-Sheet

Wichtige Endpunkte

Zweck	Pfad
Discovery	`/realms/<realm>/.well-known/openid-configuration`
Authentifizierung	`/realms/<realm>/protocol/openid-connect/auth`
Token	`/realms/<realm>/protocol/openid-connect/token`
UserInfo	`/realms/<realm>/protocol/openid-connect/userinfo`
Logout	`/realms/<realm>/protocol/openid-connect/logout`
JWKS (Schlüssel)	`/realms/<realm>/protocol/openid-connect/certs`
Account Console	`/realms/<realm>/account`
Admin REST API	`/admin/realms/<realm>`
Health / Metrics	Port 9000: `/health`, `/metrics`

Wichtige Umgebungsvariablen

Variable	Zweck
`KC_BOOTSTRAP_ADMIN_USERNAME` / `_PASSWORD`	Initialer Admin
`KC_HOSTNAME`	Öffentliche URL
`KC_HTTP_ENABLED` / `KC_PROXY_HEADERS`	Betrieb hinter Reverse Proxy
`KC_DB` / `KC_DB_URL` / `KC_DB_USERNAME` / `KC_DB_PASSWORD`	Datenbank
`KC_HEALTH_ENABLED` / `KC_METRICS_ENABLED`	Health/Metrics (Port 9000)
`KC_LOG_LEVEL` / `KC_LOG_CONSOLE_OUTPUT`	Logging
`KC_FEATURES` / `KC_FEATURES_DISABLED`	Features
`KC_CACHE` / `KC_CACHE_STACK`	Cluster-Cache

kcadm.sh — Spickzettel

```
# Anmelden
kcadm.sh config credentials --server http://localhost:8080 \
  --realm master --user admin --password admin

# Benutzer anlegen + Passwort setzen
```

```
kcadm.sh create users -r <realm> -s username=max -s enabled=true
kcadm.sh set-password -r <realm> --username max --new-password secret

# Rolle anlegen und zuweisen
kcadm.sh create roles -r <realm> -s name=trainer
kcadm.sh add-roles -r <realm> --uusername max --rolename trainer

# Realm exportieren
kc.sh export --dir /tmp/export --realm <realm>
```

Kapitel 17. Lösungsvorschläge

Dieses Kapitel enthält Lösungsvorschläge zu den Übungen aus den vorangegangenen Kapiteln. Oft gibt es neben der vorgeschlagenen Lösungen weitere Lösungsalternativen, die ebenfalls richtig sind.

Lösung zu Übung 01

- `docker-compose.yml` mit `image: quay.io/keycloak/keycloak:26.8.1` und `command: start-dev`
- Variablen `KC_BOOTSTRAP_ADMIN_USERNAME` / `KC_BOOTSTRAP_ADMIN_PASSWORD` setzen, Port `8080:8080` mappen
- Starten mit `docker compose up`
- `http://localhost:8080/admin` aufrufen und mit admin/admin anmelden

Lösung zu Übung 02

- `docker-compose.yml` und `tls.yml` aus `uebung02` verwenden
- Datei `.env` editieren und `KEYCLOAK_HOST` passend setzen.
- Zertifikat mit `generate_cert.sh` für den VM-Hostnamen erzeugen
- Private Key nach `certs/privkey.pem`, Zertifikatskette nach `certs/cert.pem` kopieren (mit `sudo`)
- `docker compose up`, dann die Admin-Console per HTTPS unter dem VM-Hostnamen öffnen und das Zertifikat prüfen

Lösung zu Übung 03

- `docker-compose.yml` aus `uebung03` nutzen (`KC_DB=postgres` plus `db`-Service)
- In `.env` die Hostnamen anpassen
- `docker compose up`
- DB prüfen: `docker compose exec db psql -U keycloak`, `\dt` zeigt u.a. `user_entity`

Lösung zu Übung 04

- Discovery-Endpunkt `/realms/master/.well-known/openid-configuration` aufrufen
- Dort `authorization_endpoint`, `token_endpoint` und `jwks_uri` ablesen
- `jwks_uri` (`.../protocol/openid-connect/certs`) liefert die öffentlichen Signatur-Schlüssel (JWKS), mit denen Tokens validiert werden

Lösung zu Übung 05

- "Create Realm" → Name `training`
- Realm settings → Login: "Forgot password" und "Remember me" aktivieren
- Realm settings → Email: Host `mailcatcher`, Port `1025`
- Realm settings → Events (User- und Admin-Events): Speichern aktivieren, Expiration je 30 Tage

Lösung zu Übung 06

- Realm settings → Login: "User registration" und "Verify email" aktivieren
- Authentication → Policies → Password policy: "Not Username", "Not Email", "Minimum Length" = 10, "Digits" = 1
- Über die Account Console registrieren; die Bestätigungsmail erscheint im Mailcatcher

Lösung zu Übung 07

- Realm roles → Create role: `trainer` und `student`
- Users → Add user: Klaus Klöbner und Hermann Wohlgemut; Credentials: temporäres Passwort (Temporary = On)
- Groups → Create group `Trainer`, Role mapping → Assign role `trainer`
- Klaus als Member der Gruppe hinzufügen → er erhält `trainer` über die Gruppe

Lösung zu Übung 08

- User federation → Add Ldap providers, Vendor: Active Directory
- Connection URL `ldap://ad1`, Bind DN `CN=Administrator,CN=Users,DC=ad1,DC=example,DC=com`, Bind credentials `Passw0rd`
- "Sync Registrations" auf Off, speichern, "Test connection" / "Test authentication"
- Prüfen über Benutzersuche in der Admin Console und Login in die Account Console

Lösung zu Übung 09

- Identity providers → Add provider → GitHub
- Client ID und Client Secret eintragen (eigene OAuth-App oder vorgegebene Daten)
- Keycloaks "Redirect URI" als "Authorization callback URL" in der GitHub-OAuth-App hinterlegen
- In der Account Console über "Sign in with GitHub" anmelden

Lösung zu Übung 10

- Realm settings → Organizations auf On
- Manage → Organizations → Create: Name `trion`, Alias `trion`, Domain `trion.de`

- Im Tab "Members" einen Realm-Benutzer hinzufügen oder per "Invite member" einladen
- Einladungsmail im Mailcatcher prüfen

Lösung zu Übung 11

- Clients → Create client: Client ID `test-app`, Client authentication = Off (public)
- Valid redirect URIs: `https://www.keycloak.org/app/*`, Web origins: `https://www.keycloak.org`
- In der Test-App Keycloak-URL, Realm und Client-ID eintragen und anmelden
- Bonus: Access Token auf jwt.io einfügen und die Claims betrachten

Lösung zu Übung 12

- Users → Benutzer → Required actions "Configure OTP" setzen
- In der Account Console anmelden, unter "Account security / Signing in" einen TOTP-Generator (z.B. FreeOTP) per QR-Code registrieren
- Nach Ab- und erneuter Anmeldung wird zusätzlich das One-Time-Password abgefragt

Lösung zu Übung 13

- Authentication → Flows → Browser-Flow duplizieren, Schritt "WebAuthn Passwordless Authenticator" ergänzen, "Bind flow"
- Neuen Benutzer mit Required Action "Webauthn Register Passwordless" anlegen
- In Chrome den virtuellen WebAuthn-Authenticator aktivieren (DevTools)
- In der Account Console registrieren und passwortlos anmelden

Lösung zu Übung 14

- `themes/demo/login/theme.properties` mit `parent=keycloak.v2` anlegen
- CSS-Datei unter `resources/css/` mit neuem Hintergrundbild (`.login-pf body`), in `styles` eintragen
- Theme-Verzeichnis nach `/opt/keycloak/themes` mounten
- Im Realm unter "Themes" als Login-Theme auswählen (Referenz: `uebung14`)

Lösung zu Übung 15

```
kcadm.sh config credentials --server http://localhost:8080 \
  --realm master --user admin --password admin
kcadm.sh create users -r training -s username=max -s enabled=true
kcadm.sh set-password -r training --username max --new-password secret
kcadm.sh get users -r training
```